Language Theory



To understand how to express information and computation, it is useful to have a formal theory of what a language is.

CONTENTS

Why? • Preliminaries • Symbols • Alphabets • Strings • Formal Languages • Language Generation • Language Recognition • Formal Language Classification • Semantics • Programming Language Theory (PLT) • Recall Practice • Summary

Why?

Why would we ever even want a theory of languages?

Languages are all about how we represent and communicate information and computations. A formal theory of language allows us to rigorously define what language is, and from that we can show exactly what various languages can represent and say.

Preliminaries

Theories of computation, including language theory, start from the idea that information can be represented as a string of symbols.

Information is that which informs. In information theory, it is the resolution of uncertainty. The more you know, the less uncertain you are.

Information theory and language theory both use a fair amount of logical and mathematical notation. If you need to brush up, see these logic notes and these math notes.

Symbols

A **symbol** is a primitive unit of data representation. Information can be represented with **strings** of symbols. For example:

- `91772.3e-8`
- `Stay behind the yellow line!`
- `ᐊᐃᖏᓗᐃ ᓄᓇᕐᕎᐊᖅ`
- `/Pecs/Los Angeles/Berlin/Madrid//1 2/3 0/2 2/2 0/3 2///`
- `(* (+ 4 3) (-9 7))`
- `int average(int x, int y) {return (x + y) / 2;}`
- `<dog id="30024"><name>Lisichka</name><age>13</age></dog>`
- `{"type": "dog", "id": "30024", "name": "Lisichka", "age": 13}`
- `∀α β. α⊥β ⇔ (α•β = 0)`
- `1. f3 e5 2. g4 ♕h4++`

> ***Exercise**: Do you buy it? Can you think of any information that cannot be represented with a string of symbols? What about images? Sounds? Thoughts? Ideas? It's okay to get philosophical and debate!*

Alphabets

Symbols are chosen from an **alphabet**, such as $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ for the language of natural numbers, or $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, ., \mathrm{e}, \mathrm{E}, +, -\}$ for the language of floating-point numbers.

Here are some examples of alphabets:

- $\mathsf{BinaryDigits} = \{0, 1\}$
- $\mathsf{DecimalDigits} = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
- $\mathsf{HexDigits} = \mathsf{DecimalDigits} \cup \{a, b, c, d, e, f, A, B, C, D, E, F\}$
- $\mathsf{Suits} = \{\spadesuit, \heartsuit, \diamondsuit, \clubsuit\}$

- PropositionalLogicSymbols $= \{p, q, r, \prime, (,), \neg, \wedge, \vee, \supset, \equiv\}$
- Unicode $= \{\sigma \mid \sigma \text{ is a Unicode character}\}$

Strings

A **string** is a finite sequence of symbols over some alphabet.

> **Examples**: For the alphabet $\{a, b, c\}$, three example strings are: $abbcbabb$, b, aaa.

When talking about strings, we might use variables to represent them. In this case the variable names should be, necessarily, outside the alphabet:

> **Examples**: $w = abbcbabb$, $x = b$, $y = aaa$.

The **length** of a string is the number of *symbol occurrences* in the string.

> **Example**: if $w = abbac$, then $|w| = 5$.

The **empty string**, denoted ε, is the (unique) string of length 0.

The **concatenation** of two strings x and y is the string obtained by appending y to the end of x. We have a notation for repeated concatenation: $w^0 = \varepsilon$, $w^1 = w$, $w^2 = ww$, and $w^{k+1} = ww^k$.

> **Example**: if $x = abb$ and $y = ca$ then $xy = abbca$.
>
> **Example**: $w\varepsilon = \varepsilon w = w$.
>
> **Example**: $(abc)^4 = abcabcabcabc$.

> ***Tricky Notation?***
>
> *String concatenation looks like numeric multiplication, and string repetition looks like numeric exponentiation, but behaviorally, concatenation is like numeric addition and repetition is like numeric multiplication. If you've programmed in Python, you'll see that is indeed true.*
>
> ```
> $ python
> >>> "abc" + "def"
> 'abcdef'
> >>> "abc" * 3
> 'abcabcabc'
> ```

Guido got it right.

The **reversal** w^R of a string w is the string formed by reversing w; formally $\varepsilon^R = \varepsilon$ and $(\sigma w)^R = w^R \sigma$ for some symbol σ.

x is a **substring** of y iff $\exists v.\ \exists w.\ y = vxw$.

Because we are defining languages with an eye toward modeling computation, we almost always require alphabets to be non-empty and finite, and require all strings to have a finite (but unbounded) length. Even with a finite alphabet and finite-length strings, you can still have an infinite number of strings!

> **Example**: The finite alphabet $\{0, 1\}$ gives rise to the infinite number of strings 0, 1, 00, 01, 10, 11, 000, 001, etc.

Formal Languages

A **formal language** is a set of strings over some alphabet. That's it. That's the definition. If a string w is a member of a language L we say w is an **utterance** of L.

Examples

Examples first! Here are some example languages:

- $\{w \in \{a, b\}^* \mid w \text{ has the same number of } a\text{'s as } b\text{'s}\}$
- $\{w \in \{a, b\}^* \mid w \text{ has more } a\text{'s than } b\text{'s}\}$
- $\{w \in \{a, b\}^* \mid \text{the first and last symbol of } w \text{ are the same}\}$
- $\{a^n b^n \mid n \geq 0\}$
- $\{a^i b^j \mid j = 2^i\}$
- $\{a^i b^j c^k \mid k = i + j\}$
- $\{ww \mid w \in \{a, b\}^*\}$
- $\{ww^R \mid w \in \{a, b\}^*\}$
- $\{1^n \mid n \text{ is prime}\}$
- $\{w \in \{0, 1\}^* \mid w \text{ is a binary numeral greater than } 55\}$
- $\{w \in \{0, 1\}^* \mid w \text{ is a binary numeral divisible by } 5\}$
- $\{w \in \text{DecimalDigit}^* \mid w \text{ is a valid MasterCard number}\}$

- $\{w \in \text{Unicode}^* \mid w \text{ contains the substring } dog\}$
- $\{w \in \text{Unicode}^* \mid w \text{ is a valid Python program}\}$
- $\{w \in \text{Unicode}^* \mid w \text{ is a valid JavaScript program}\}$
- $\{w \in \text{Unicode}^* \mid w \text{ is a valid Swift program}\}$
- $\{w \in \text{Unicode}^* \mid w \text{ is a valid Swift program that prints the integers from 1 to 10}\}$

***Exercise**: For each of the example languages above, give a few strings in (utterances of) the language.*

> ***Why are languages sets?***
>
> *A language as a set of strings is actually quite useful. It distinguishes strings "in" the language from those not in the language. A language is a set of (legal) utterances.*
>
> *Now it's true that this definition does not say anything about the "meaning" of each utterance. That has to be handled separately. But we are off to a good start.*

Language Operations

Because languages are sets, $\varnothing$ is a language, and the **union** $(L_1 \cup L_2)$ and **intersection** $(L_1 \cap L_2)$ of languages make perfect sense. So does the **cardinality** of a language, $|L|$, the number of strings in the language. In addition:

- The **concatenation** of two languages L_1 and L_2 is $L_1L_2 = \{xy \mid x \in L_1 \wedge y \in L_2\}$. Also, we define $L^0 = \{\varepsilon\}$, $L^1 = L$, $L^2 = LL$, ..., $L^{k+1} = LL^k$.
- The **Kleene Closure** of L is $L^* = L^0 \cup L^1 \cup L^2 \cup \ldots = \{w_1w_2 \ldots w_k \mid w_i \in L \wedge k \geq 0\}$.
- The **Positive Closure** of L is $L^+ = L^1 \cup L^2 \cup L^3 \cup \ldots = \{w_1w_2 \ldots w_k \mid w_i \in L \wedge k > 0\}$.

Therefore, given an alphabet Σ, the following are languages over Σ:

- $\varnothing$, the empty language, the language of no strings
- $\{\varepsilon\}$, the language containing the empty string and nothing else
- Σ, the language of all possible one-symbol strings
- Σ^2, the language of all possible two-symbol strings
- Σ^3, the language of all possible three-symbol strings
- Σ^*, the language of all possible strings over the alphabet Σ

> ***The Infinite Library***
>
> *The language Σ^* (the language of all possible strings over an alphabet) is fascinating. It's basically the infinite library, which contains every book that has ever been written and that could possibly ever be written, i.e., every single possible arrangement of symbols. It contains solutions to climate change and world hunger, the cures for all cancers, and all human histories and futures (yours included). It also contains an infinite amount of misinformation. The problem is, you can't distinguish what is right from wrong.*
>
> *When you "know" everything, you know nothing.*

For any language L over the alphabet Σ, we have $L \subseteq \Sigma^*$. Since we *must* specify an alphabet for each language, the **complement** of a language makes sense. It is defined as follows: $\overline{L} = \Sigma^* - L$

Let's see the language operations in use. Suppose we are given the alphabet $\Sigma = \{a, b\}$, and two languages over Σ, which we'll define as $L_1 = \{ba, b\}$ and $L_2 = \{abb, bb\}$.

Then:

- $\Sigma^* = \{\varepsilon, a, b, aa, ab, ba, bb, aaa, aab, aba, abb, baa, bab, \dots\}$
- ${L_1}^* = \{\varepsilon, ba, b, baba, bab, bba, bb, \dots\}$
- ${L_1}^+ = \{ba, b, baba, bab, bba, bb, \dots\}$
- $L_1 L_2 = \{baabb, babb, bbb\}$
- $L_1 \cup L_2 = \{ba, b, abb, bb\}$
- $L_1 \cap L_2 = \varnothing$

Random Language Facts

Here are some interesting theorems about languages. Ready? Let Σ be an alphabet and A, B, C, and D be languages over Σ (i.e., subsets of Σ^*). Then:

- $A\varnothing = \varnothing A = \varnothing$
- $A\{\varepsilon\} = \{\varepsilon\}A = A$
- $(AB)C = A(BC)$
- $(A \subseteq B) \wedge (C \subseteq D) \Rightarrow AC \subseteq BD$
- $A(B \cup C) = AB \cup AC$
- $(B \cup C)A = BA \cup CA$
- $A(B \cap C) \subseteq AB \cap AC$

- $(B \cap C)A \subseteq BA \cap CA$

- $A^* = A^+ \cup \{\varepsilon\}$
- $n \geq 0 \Rightarrow A^n \subseteq A^*$
- $n \geq 1 \Rightarrow A^n \subseteq A^+$
- $A \subseteq AB^*$
- $A \subseteq B^*A$
- $A \subseteq B \Rightarrow A^* \subseteq B^*$
- $A \subseteq B \Rightarrow A^+ \subseteq B^+$
- $AA^* = A^*A = A^+$
- $\{\varepsilon\} \in A$ iff $A^+ = A^*$
- $(A^*)^* = A^*A^* = A^*$
- $(A^*)^+ = (A^+)^* = A^*$
- $A^*A^+ = A^+A^* = A^*$
- $(A^*B^*)^* = (A \cup B)^* = (A^* \cup B^*)^*$

- $(C = AC \cup B) \Rightarrow (C = A^*B)$

> ***Exercise**: For each of the above theorems, try to understand what they are saying and why they are true. If you are motivated, try to write proofs for each. They aren't super hard to prove, provided you know some basic Set Theory.*

Language Definition

Formal languages were developed to help us study the foundations of mathematics and computation. It's true that in this context, languages are just sets, so we can communicate them in set notation, for example:

- $\{w \mid w \textbf{ is a legal floating point numeral}\}$

But wait, even though that definition used mathematical symbols in part, there was a big informal piece between the curly braces. This can be a problem. Informal definitions lack precision. Like what even are floating point numerals? We can list as many examples as possible to get the idea. Maybe we can list them all, say something like { 0, 8, 3.5, 13.55, 3E8, 3.2E89, 5.111E+2, 21.34E-2, 1e-8, 2.55e+233, ...}. This is also not precise. Let's try harder, with a definition:

- The set of all strings that begin with a sequence of one or more decimal digits,
  - optionally followed by a dot then one or more decimal digits
  - optionally followed by:

- an `E` or an `e`
- then, optionally, a `+` or `-`
- then a sequence of one or more decimal digits

Also not very satisfying: it is very verbose, kind of confusing, and hard to tell if it is even right. It is "precise" *but it is still informal*! We need something better. Something formal. As it happens, we can do this! There are two major approaches:

Definition by Generation

We can give a formalized set of rules that systematically generate all and only those strings in the language. The language is then the set of all string **generated** by the language.

Definition by Recognition

We can give a formal definition of a process that determines, for a given string, whether or not it is in the language. The language is then the set of all strings **recognized** by the process.

Language Generation

The primary mechanism to define a language via generation is the generative grammar.

Generative Grammars

A **generative grammar** is a list of **rules** that generates all and only those strings in a particular language. The rules have **variables** that get replaced in order to generate the actual strings of the language. Here's a grammar for our floating point numeral language. The alphabet for this language is $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, ., E, e, +, -\}$. Our grammar will use four variables, $\{\textit{numeral}, \textit{fractionalPart}, \textit{exponentPart}, \textit{digit}\}$:

numeral	$\longrightarrow$	*digit*$^+$ *fractionalPart*? *exponentPart*?
fractionalPart	$\longrightarrow$	`"."` *digit*$^+$
exponentPart	$\longrightarrow$	(`"E"`|`"e"`) (`"+"`|`"-"`)? *digit*$^+$
digit	$\longrightarrow$	`"0".."9"`

There are multiple grammar notations out there; the one used here features the following notation:

- → means “can be replaced with”
- `+` means “one or more”
- `*` means “zero or more”
- `?` means “optional”
- `|` means “or”
- Double quotes delimit the actual symbols in the language you are defining, while variables are unquoted
- Parentheses are for grouping
- `..` makes a range over individual symbols (yeah, it’s kind of shorthand)

If G is a grammar, then $L(G)$ is the language defined by the grammar G. A string is in $L(G)$ if it is **derivable** from G’s initial rule, repeatedly replacing instances of a rule’s left hand pattern with its right hand side, until you get completely rid of the variables and are left only with strings in the language’s alphabet. Here is an example **derivation**:

$$
\begin{aligned}
\textit{numeral} &\Longrightarrow \textit{digit}^+\ \textit{fractionalPart}?\ \textit{exponentPart}? \\
&\Longrightarrow \texttt{338}\ \textit{fractionalPart}?\ \textit{exponentPart}? \\
&\Longrightarrow \texttt{338.}\ \textit{digit}^+\ \textit{exponentPart}? \\
&\Longrightarrow \texttt{338.253}\ \textit{exponentPart}? \\
&\Longrightarrow \texttt{338.253}\ (\texttt{"E"}|\texttt{"e"})\ (\texttt{"+"}|\texttt{"-"})?\ \textit{digit}^+ \\
&\Longrightarrow \texttt{338.253e}\ (\texttt{"+"}|\texttt{"-"})?\ \textit{digit}^+ \\
&\Longrightarrow \texttt{338.253e}\ \textit{digit}^+ \\
&\Longrightarrow \texttt{338.253e18}
\end{aligned}
$$

***Exercise**: Show that the following strings are in the language defined by the grammar above:*

- *3*
- *8.5501e-88*
- *5E+11*

and that the following are not in the language:

- *-2.3E1*
- *55.e+2*
- *^^Dog!*

Also, generate three of your own strings in the language and three strings not in the language.

***Exercise**: Make sure you understand the difference between $\longrightarrow$ and $\Longrightarrow$.*

By the way, note that there can be many grammars that specify the same language. For example, here's another grammar for the floating-point numeral language:

numeral	⟶	*digit*$^+$ ("." *digit*$^+$)? (("E"|"e") ("+"|"−")? *digit*$^+$)?
digit	⟶	"0".."9"

If the first grammar above was called G_1 and this grammar was called G_2, we would have $L(G_1) = L(G_2)$.

Practice with Grammars

If you think about it, coming up with a grammar for a language you have in mind is an endeavor similar to programming: you know what you want to end up with, but you need to create the very formal, precise steps to get you there. How do you get good at coming up with grammars? Certainly it helps to have a mentor and learn tricks of the trade over time with practice. Whether you have a mentor or not, it does help to start with a whole bunch of examples. So with that in mind, here are a handful of grammars for you to study:

The language of integer binary numerals divisible by 2:

even	⟶	"−"? ("0" | "1")* "0"

The language of integer decimal numerals divisible by 5:

divisibleByFive	⟶	"−"? *digit** ("0" | "5")
digit	⟶	"0".."9"

The language $\{\varepsilon, 0, 1, 01, 10, 010, 101, 0101, 1010, \ldots\}$ of alternating zeros and ones:

alternating	⟶	*startsWithZero* | *startsWithOne*
startsWithZero	⟶	ε | "0" *startsWithOne*
startsWithOne	⟶	ε | "1" *startsWithZero*

The language $\{0^i 1^j \mid i \text{ is even} \wedge j \text{ is odd}\}$:

evenZerosThenOddOnes	⟶	("00")* "1" ("11")*

The language $\{a^n b^n \mid n \geq 0\} = \{\varepsilon, ab, aabb, aaabbb, aaaabbbb, \ldots\}$:

anbn	⟶	ε | "a" *anbn* "b"

The language of palindromes over $\{a, b\}$:

pal	⟶	ε | `"a"` | `"b"` | `"a"` *pal* `"a"` | `"b"` *pal* `"b"`

The language of parentheses, curly braces, and square brackets, in which everything is properly balanced and nested, for example `()[{()}[[]]()]`:

balanced	⟶	(`"("` *balanced* `")"` | `"["` *balanced* `"]"` | `"{"` *balanced* `"}"`)*

The language $\{a^n b^n c^n \mid n \geq 0\}$:

s	⟶	ε | `"a"` *s*? *x* `"c"`	
`"c"` *x*	⟶	*x* `"c"`	—move *c*'s to the right of the *x*'s
`"a"` *x*	⟶	`"ab"`	—only make a *b* after an *a* ...
`"b"` *x*	⟶	`"bb"`	—... or after a *b*

The language $\{a^{2^n} \mid n \geq 0\}$:

aToTwoToN	⟶	*double** `"a"` *right*	—mark the end
double `"a"`	⟶	`"aa"` *double*	—move right, doubling *a*'s as you go
double right	⟶	*right*	—got to end, no more to double
right	⟶	ε	

The language $\{ww \mid w \in \{a, b\}^*\}$:

s	⟶	`"a"` *s x* | `"b"` *s y* | *middle*	
middle x	⟶	*middle* `"a"`	—an *x* generates an *a*
middle y	⟶	*middle* `"b"`	—a *y* generates a *b*
`"a"` *x*	⟶	*x* `"a"`	—move *a*'s to end
`"a"` *y*	⟶	*y* `"a"`	
`"b"` *x*	⟶	*x* `"b"`	—move *b*'s to end
`"b"` *y*	⟶	*y* `"b"`	
middle	⟶	ε	—drop midmarker when no longer needed

Exercise: *Give a grammar for the language of unary primes:* $\{1^p \mid p \text{ is prime}\}$

Why are we studying such silly languages?

Obviously we need to get to the point where we can design grammars for real programming languages. But studying the fundamentals of language and grammars with bite-size pieces

helps us uncover patterns that we can use in designing our large grammars. It also gives insights into what is easy and what is hard. For example: some grammars above looked very straightforward and some required bizarre "marker" symbols that were passed around to..."compute" something it seems. The scientist in you should wonder if distinctions like that are fundamental in some sense, and if so, how can we capture this state of affairs formally.

CLASSWORK

We are going to do lots of derivations for many of the grammars above.

More Example Grammars

You can't have too many examples, so let's see more. To help us out a bit, we're going to take the following variables a being previously defined:

- ***digit***, for `"0".."9"`
- ***hexDigit***, for ***digit*** | `"A".."F"` | `"a".."f"`
- ***letter***, for any Unicode letter
- ***alnum***, for ***letter*** | ***digit***

Here is the language of words starting with a letter and containing letters, digits, and underscores only:

id	⟶	***letter*** (***alnum*** | `"_"`)*

The language of simple hex colors in HTML: Hash followed by 3 or 6 hex digits:

simpleHexColor	⟶	`"#"` ***threeHexDigits threeHexDigits?***
threeHexDigits	⟶	***hexDigit hexDigit hexDigit***

Now let's create a language for integer expressions with addition, subtraction, multiplication, division, exponentiation, negation, and parentheses. When languages start to get complex, it helps to see examples and non-examples:

Examples	Non-Examples
432 24*(31/899+3-0)/(54/-2+(5+2)*3)	43 2 24*(31////)/(5+---+))

(2) 8*(((3-6)))	[fwe]23re31124efr$#%^@ --2--

Here is a first pass. We say: "An expression is either (1) an integer, (2) a negated expression, (3) two expressions with a binary operator between them, or (4) a parenthesized expression." This gives us:

$$
\begin{array}{lcl}
\mathit{exp} & \longrightarrow & \mathit{digit}^{+} \\
& | & \texttt{"-"}\ \mathit{exp} \\
& | & \mathit{exp}\ (\texttt{"+"}\ |\ \texttt{"-"}\ |\ \texttt{"*"}\ |\ \texttt{"/"}\ |\ \texttt{"\%"}\ |\ \texttt{"**"})\ \mathit{exp} \\
& | & \texttt{"("}\ \mathit{exp}\ \texttt{")"}
\end{array}
$$

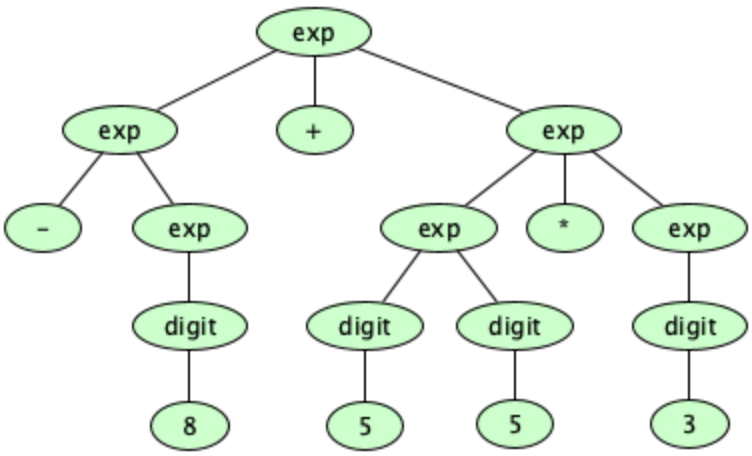
How did we do? Let's see if we can derive something interesting:

$$
\begin{array}{lcl}
\mathit{exp} & \Longrightarrow & \mathit{exp}\ \texttt{+}\ \mathit{exp} \\
& \Longrightarrow & \texttt{-}\ \mathit{exp}\ \texttt{+}\ \mathit{exp} \\
& \Longrightarrow & \texttt{-}\ \mathit{digit}\ \texttt{+}\ \mathit{exp} \\
& \Longrightarrow & \texttt{-8+}\ \mathit{exp} \\
& \Longrightarrow & \texttt{-8+}\ \mathit{exp}\ \texttt{*}\ \mathit{exp} \\
& \Longrightarrow & \texttt{-8+}\ \mathit{digit}\ \mathit{digit}\ \texttt{*}\ \mathit{exp} \\
& \Longrightarrow & \texttt{-8+5}\ \mathit{digit}\ \texttt{*}\ \mathit{exp} \\
& \Longrightarrow & \texttt{-8+55*}\ \mathit{exp} \\
& \Longrightarrow & \texttt{-8+55*}\ \mathit{digit} \\
& \Longrightarrow & \texttt{-8+55*3}
\end{array}
$$

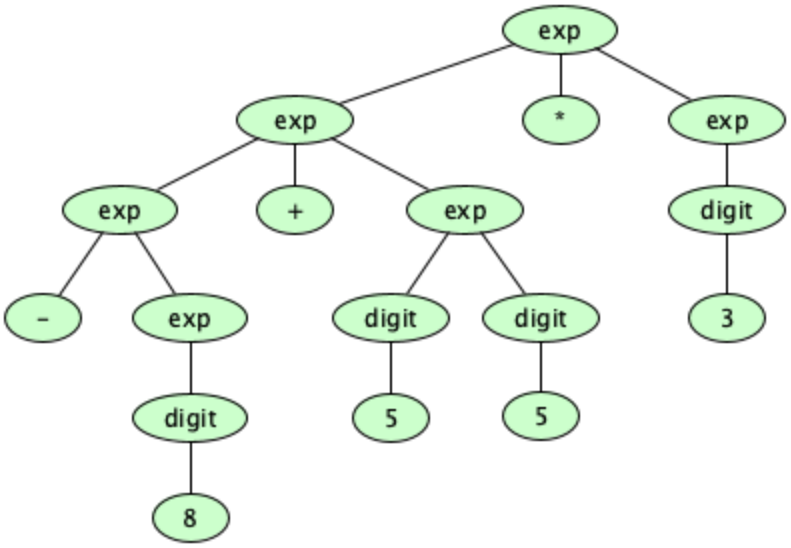
Derivations can get pretty long. Is there a better way to see how our strings are made?

Derivation Trees

Here's a better way to "show" the derivation we just performed:



But wait! There can be more than one way to derive a string, and sometimes (**but not always**) the resulting **derivation tree** is different for different derivations:



> ***Exercise***: *Show the (long-form) derivation of the string corresponding to the second tree above.*

> ***Exercise***: *Give at least two derivation trees for* `-5*(34/-55+(21-8)+233)`. *How many distinct trees are there for this string?*

Is having multiple derivation trees for a string a big deal?

Ambiguity

A grammar is **ambiguous** if there exists more than one derivation tree for a given string. Natural language grammars are often ambiguous.

> ***Exercise***: *Show that the following sentences, possibly derived from a grammar for English, are ambiguous.*
>
> - *Lawyers give poor free advice.*
> - *Teacher strikes idle kids.*
> - *Seven foot doctors sue hospital.*
> - *Stolen painting found by tree.*
> - *Enraged cow injures farmer with ax.*
> - *Two sisters reunited after 18 years at checkout counter.*
> - *The train left the station crowded and dirty.*

- *Landmine claims dog arms company.*
- *Hershey bars protest.*
- *They cooked for the first time on the stove.*

Try your best to show reasonable English grammar "trees."

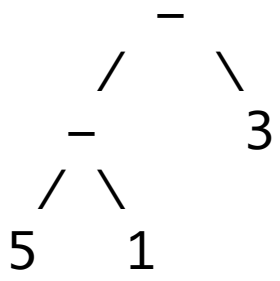
But for a programming language, we don't really like ambiguity, since the derivation tree tells us the structure of a program (its **syntax**) from which we wish to assign a meaning (**semantics**). Under the widely accepted meanings of decimal integers and arithmetic operations, those two trees we saw earlier suggest meanings of 157 (multiply-then-add) and 141 (add-then-multiply). Yikes!

One way to get rid of ambiguity in our language is to force operator **precedence** and **associativity** in the grammar. The following convention has arisen:

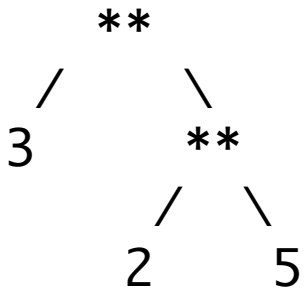
- Expressions are terms separated by left-associative additive operators (`+`, `-`)
- Terms are factors separated by left-associative multiplicative operators (`*`, `/`, `%`)
- Factors are primaries separated by right-associative exponentiation operators (`**`)
- Primaries are numerals, negated primaries, or parenthesized expressions.

Because expressions are made up of terms and terms are made up of factors, the utterance `1+3*2` is made up of two terms, `1` and `3*2`. Therefore we say multiplication has **higher precedence** than addition. This is the only way to make the derivation tree! There is no ambiguity here. Now what about `(1+3)*2`? This expression has a single term, which has two factors: `(1+3)` and `2`. Note how the parentheses matter!

But what about the utterance `5-1-3`? Should this be structured to do the leftmost subtraction first (yielding 1) or the rightmost first (yielding 7)? This is where associativity comes in. Tradition says subtraction is **left-associative**, so we want a structure with:



Exponentiation is traditionally **right-associative**, so `3**2**5` would desire a derivation structure as:



Do you know what else might be ambiguous? Expressions like `-2**2`. In Python, exponentiation is done before negation, so this expression produces `-4`. In Elm, negation happens first, so the expression evaluates to 4. JavaScript uses a grammar that prevents the expression from being derived by the grammar! This is a good thing. Let's do it.

We are now ready to give the **non-ambiguous** grammar for our integer arithmetic expression language:

exp	⟶	*exp* ("+" | "−") *term*
	|	*term*
term	⟶	*term* ("*" | "/" | "%") *factor*
	|	*factor*
factor	⟶	*primary* ("**" *factor*)?
	|	"−" *primary*
primary	⟶	*numeral*
	|	"(" *exp* ")"
numeral	⟶	$digit^+$

CLASSWORK

We are going to make a long-ish expression and show that it has one and only one derivation tree.

***Exercise**: Show that `-3**2` cannot be derived from this grammar.*

The Formal Definition of a Grammar

We've seen that we use grammars to formally describe mathematical objects we call languages. But we were kind of hand-wavy in defining what a grammar is and how derivations work. Grammars are mathematical objects themselves and so can be described formally. Here is how:

> A **grammar** is a quadruple (V, Σ, R, S) where:

- Σ is the alphabet of the language being described
- V is a set of **variables**, such that $\Sigma \cap V = \varnothing$
- $R \subseteq (V \cup \Sigma)^* V (V \cup \Sigma)^* \times (V \cup \Sigma)^*$ is a set of rules
- $S \in V$ is the start variable

The formal definition of R looks complex but it really just says "the left hand side must contain at least one variable." That is all. Super reasonable too! It's there so once you get rid of all the variables, you don't keep going.

Example. The grammar shown earlier for the language

$$\{0^n 1^n \mid n \geq 0\}$$

is *formally* denoted

$$(\{s\}, \{0, 1\}, \{(s, \varepsilon), (s, 0s1)\}, s)$$

Please work through this to make sure you "see" it and understand it.

***Exercise**: Give the formal representation of the grammars from the previous section.*

Derivations can be formalized too:

> Given any grammar rule set R, there exists a corresponding **derivation relation** $\Longrightarrow_R$ defined as $\{(xwy, xzy) \mid wRz\}$. That is, $xwy \Longrightarrow_R xzy$ **iff** wRz.

As can the language defined by a grammar:

> The **language defined by** grammar (V, Σ, R, S) is $\{\sigma \in \Sigma^* \mid S \Rightarrow_R^* \sigma\}$

We also use the notation $L(G)$ for the language defined by grammar G.

CLASSWORK

You might be wondering about the `|` and `?` and `*` and `+` symbols and why they do not appear in our formal definition. As it turns out, they are not needed: they are just shorthands ("syntactic sugar") for proper rules. We will work out in class exactly how they can be replaced with regular rules.

Terminology

The page you are reading calls V the set of **variables** *and Σ the* **alphabet**. *You may encounter folks who call V the set of* **non-terminals** *and Σ the set of* **terminals**. *No matter what the names, $V \cap \Sigma = \varnothing$.*

Well...actually...there are still other folks that call V a **vocabulary** *and make it the set of all variables plus the alphabet symbols and consequently tell you now that $\Sigma \subset V$.*

I know, I know. The moral here is that there ain't just one way to label all the ideas behind grammars. Different authors describe this stuff differently.

Can grammars be simplified?

The development of formal languages originated with early work by Emil Post, Axel Thue, and others working in the early 20th century on questions related to the foundations of mathematics and computation. Post created a system for transforming strings, called a Post Canonical System. We now know a much simpler formulation of Post's work, namely a semi-Thue system, a.k.a. string rewriting system. An SRS can rewriting any string into any other; it is nothing more than a tuple (Σ, R) where Σ is the alphabet of symbols and $R \subseteq \Sigma^* \times \Sigma^*$ is the rewriting relation.

That's it!

However, despite their simplicity, they are rather clunky to use to define languages. Chomsky's genius move was to add variables, making a special kind of a rewriting system known as a **grammar**. *Brilliant! This is why all computer scientists encounter grammars and only very few have heard of string rewriting systems.*

Restricted Grammars

In practice, it is often super-convenient to talk about grammars with restricted rule sets, as these may yield grammars which are much simpler to understand and work with. Of course, you lose the ability to derive as many languages as an unrestricted grammar, but the simplicity might be worth it. There are many dozens of known restricted grammar types, but there are three that you should know: Context-Free, Right-Linear, and Essentially Noncontracting.

Context-Free Grammar

A **CFG** is a grammar where $R \subseteq V \times (V \cup \Sigma)^*$, that is, the left hand side is **just a single variable**. So simple and clean, sensible and intuitive. Because of this, they've been massively successively successful and the foundation for the vast majority of structural definitions for real programming languages. Interestingly, they aren't actually powerful enough to capture all legal programs in most programming languages! We'll see later how to handle this problem in practice.

> ***Why are they called context free?***
>
> *Context-free grammars are so-named because variables can be rewritten regardless of the context in which they appear. (For example, context free grammars would allow say, an expression in a programming language, to be any identifier whatsoever at any time, regardless of whether one is in the context in which the identifier was previously declared or not.)*

Right-Linear Grammar

An **RLG** is a grammar where $R \subseteq V \times (\Sigma^* \cup \Sigma^* V)$, that is, the left hand side is just a single variable and the right hand side is a string of zero or more symbols followed by at most one variable. Pretty interesting: their derivations line up with the idea of a linear computation without memory. More on this later.

Essentially Noncontracting Grammar

In every rule in a **ENCG**, the right hand side never has fewer symbols and variables than the left hand side—with only one possible exception: if the language contains the empty string, we can have one rule that maps the start symbol to the empty string. The conceptual idea for a noncontracting grammar is that the derivation string never gets shorter; the **essentially** noncontracting grammars are those that have that one exception to allow for the empty string being in the language. These correspond to computations with bounded memory.

Language Recognition

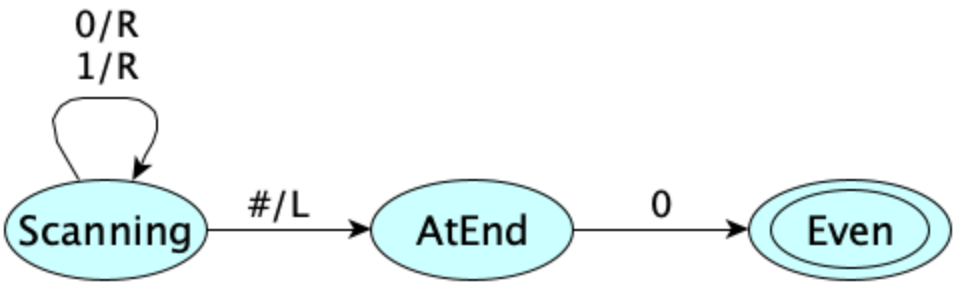
A grammar lets us formally specify a language by **generating** all and only those strings in the language. But we can define a language another way: we can give a process that, when given a string,

determines whether it belongs to a language?

Hey this is actually very cool! Such a process can be used in a compiler! Let's learn.

Automata

One way to define a language through recognition is to use automata with **accept states**. In earlier notes we saw a Turing Machine which accepted all and only those strings in the language of even binary numerals:



Both the grammar `even → ("0"|"1")*"0"` and the Turing Machine above "define" the same language, one in a generative fashion and the other in a recognitive fashion. Cool.

But there's more! Just like we showed several ways to ~~dumb-down~~ restrict grammars for the purpose of generating certain interesting classes of languages, Turing Machines can be restricted in several ways, too, each of which correspond to certain kinds of grammars. (This is either creepy, a massive coincidence, or something fundamental about information.) Languages and Automata are intimately related! For example, we can:

- Ditch the infinite tape and bound the tape to the size of the input plus one cell on either end, containing a left-end-marker-symbol on the left and a right-end-marker-symbol on the right. These cells cannot be changed nor can the TM head move off the ends. This is called a **linear bounded automaton**, (LBA). The languages they recognize are equivalent to those generated by a Type-1 Grammar.

- Restrict the machine to be read-only (never write anything) and only move right. This is called, and note this is a misleading name, a **finite automata** (FA). It is a poor name because it is not the only kind of machine with a fine number of states (all TMs have a finite number of states) and it is not the only kind with a finite memory (LBAs are bounded also). But FAs are very important: they define computations that *process one symbol at a time and don't require any scratch memory*. The recognize the same class of languages as are derived by RLGs.

- Add a second tape (called the "stack") and keep the input tape read-only and allow right-moves only. The next step is chosen based on both the current input symbol and the symbol(s) on the top of the stack, At each step you can pop symbols and push symbols. This contraption is called a **pushdown automata** (PDA). PDAs recognize exactly the context-free languages. PDAs can be restricted even further so that there is only one possible move to make at each step: this yields a "deterministic" PDA, known as a DPDA.

> ***Determinism***
>
> *Automata theory is a huge topic, and there is so much more to study. We'll get to it later. But there's one interesting aspect worth noting now: Machines that allow more than one possible operation from a given configuration are called* **nondeterministic**, *and those that allow only one are called* **deterministic**. *Okay, fine, but here's the philosophical question: is nondeterminism more "powerful" than determinism? That is, can we recognize more languages with nondeterministic machines of a given class than deterministic ones? It turns out:*
>
> - *For FAs and TMs, there is NO difference in recognition power between deterministic and nondeterministic versions.*
> - *For PDAs, there IS a difference in recognition power between deterministic and nondeterministic versions.*
> - *For LBAs, we don't know!*
>
> *Who knew right?* 🤯

Analytic Grammars

An **analytic grammar** is crafted with the idea of making recognition—going from candidate string to start symbol—much more natural. You don't think about generating all possible utterances from the start symbol; instead, you think of **matching** rules to a candidate string. This idea is pretty nice when you need to write compilers or interpreters.

The biggest difference between generative and analytic grammars is in the handling of alternatives. In a generative grammar G, $L(G)$ is contains all strings that *can* be generated by applying rules whenever possible. For example, the rule $x \rightarrow y \mid z$ means you can replace an x with an y or a z, so you can generate both. So the grammar:

$s \longrightarrow$ `"a"` | `"ab"`

describes the language $\{a, ab\}$.

In an analytic grammar, the rules are deterministic; you don't get to pick one or the other. So $A \,/\, B$ means: "Try to match A. If you can, the rule matches and you are done. Otherwise (and only otherwise), try to match B". So:

```
s ⟵ "a" / "ab"
```

recognizes the language $\{a\}$, but:

```
s ⟵ "ab" / "a"
```

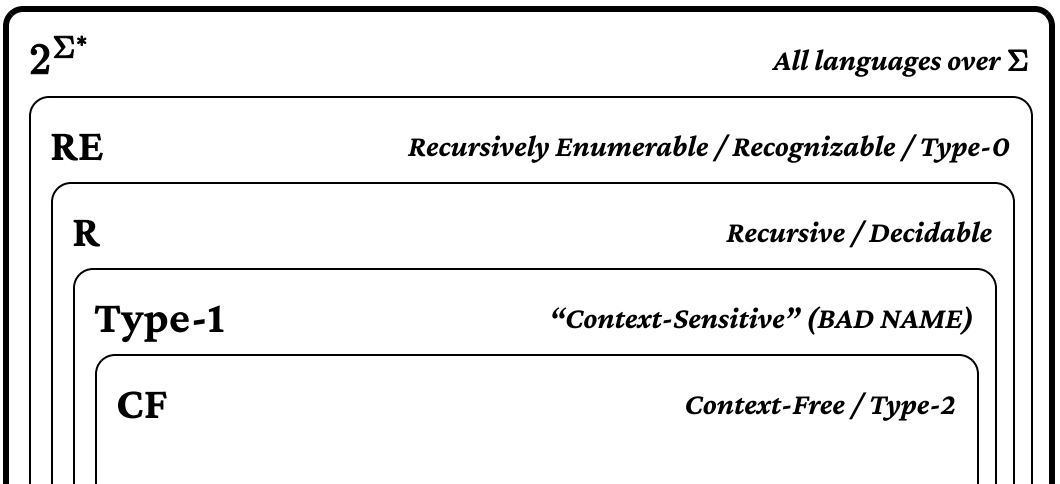
recognizes the language $\{a, ab\}$.

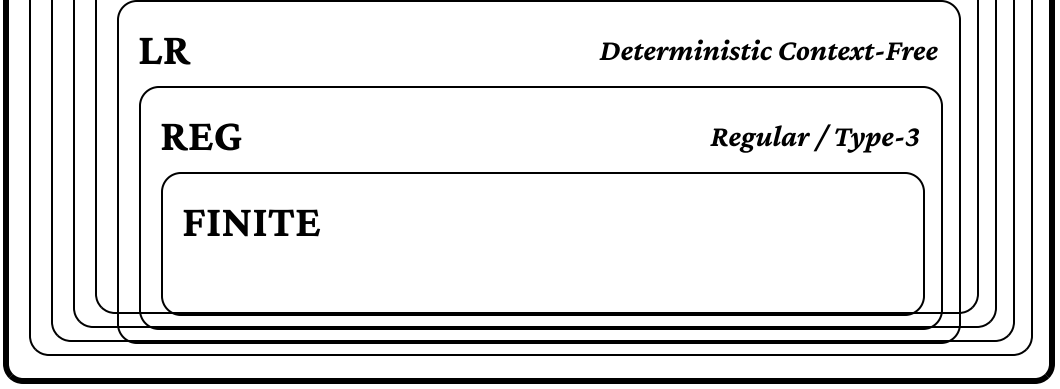
Unlike generative grammars, analytic grammars feature special **lookahead operators**, including the **negative lookahead operator** (~) and the **positive lookahead operator** (&). This enables the definition of non-context-free languages, despite having only one symbol on the left-hand side! Here's an analytic grammar for the non-context-free language $\{a^n b^n c^n \mid n \geq 0\}$:

```
s ⟵ & (x "c") "a"+ y
x ⟵ "a" x? "b"
y ⟵ "b" y? "c"
```

Formal Language Classification

Time for a little side exploration into how we can categorize languages. There are a gazillion ways to do so. Perhaps the first and certainly the most historically significant classification scheme is due to Chomsky, and is now known as the **Chomsky Hierarchy**:





The main language types in this hierarchy are listed below.

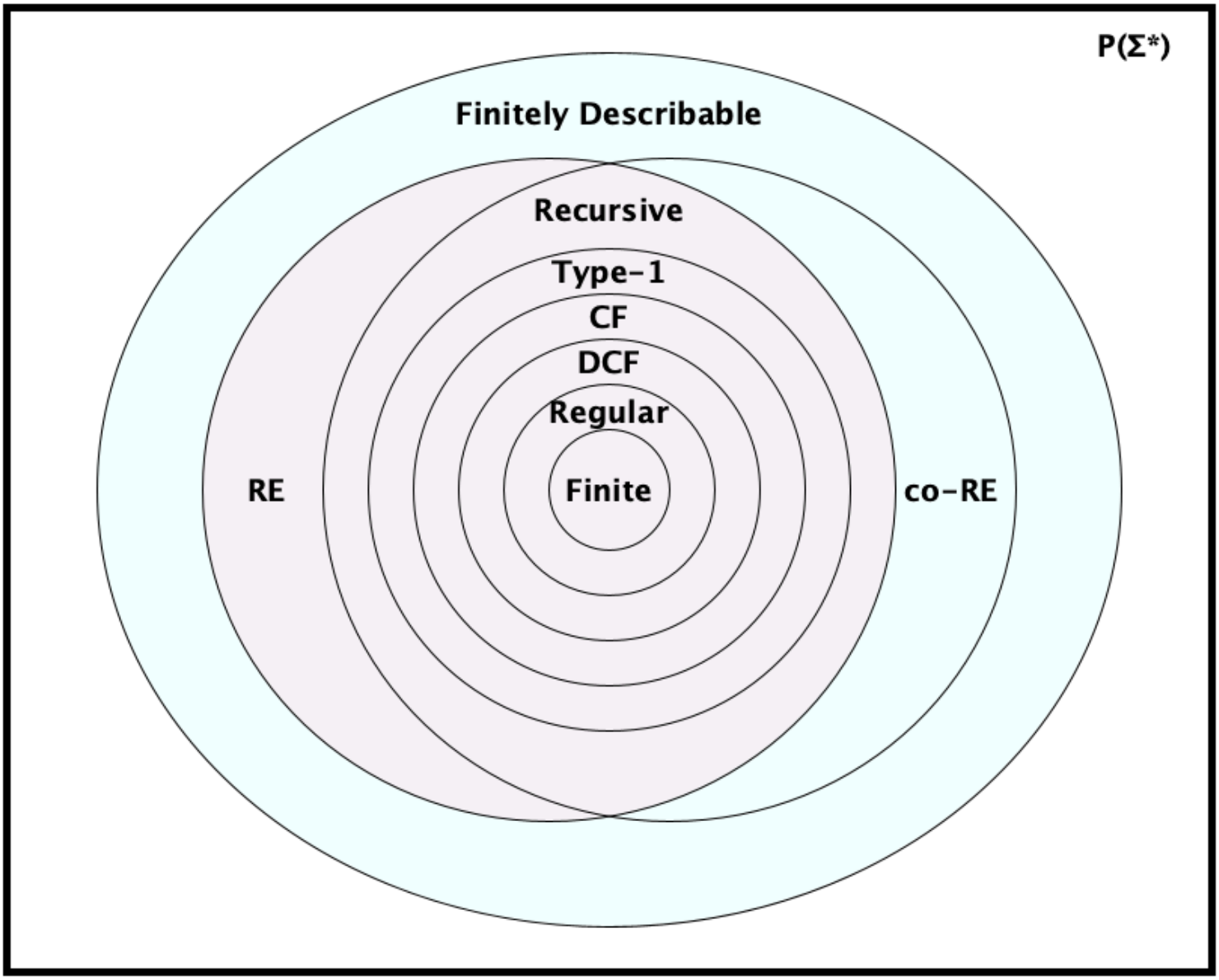
Language Type	Description	Grammar	Recognizer
Finite	A language with only a finite number of strings.	$S \to w_1 \vert \ldots \vert w_n$	Loop-free FA
Regular (Type 3)	A language in which strings can be recognized by processing them one symbol at a time, without backtracking, and **without using additional memory**.	RLG	FA
Deterministic Context Free (LR)	A language recognizable on a deterministic PDA. (Stacks are nice! $\Theta(1)$ access! Determinism is nice too! Much more efficient.)	DCFG	DPDA
Context Free (Type 2)	A language recognizable on a nondeterministic PDA. (Stacks are nice! $\Theta(1)$ access! But with nondeterminism, recognition is slower.)	CFG	PDA
Type 1	A language recognizable on an LBA. (Type 1 languages are often called "context-sensitive," but that is not a very accurate name because all languages "beyond" Type 1 in the hierarchy are sensitive to context).	ENCG	LBA
Recursive (R)	A language in which string membership is **decidable**. There are no predetermined memory bounds, and no bounds on the number of steps.	?	TM that always halts

Recursively Enumerable (Type 0 / RE)	A language in which string membership is **partially decidable**: if a string is in the language, you can determine that for sure; if a string is not, you might not be able to determine this.	(Unrestricted) Grammar	TM

We sometimes add:

- **co-RE**: A language whose complement is RE
- **Finitely Describable**: A language that has a finite description

Fun fact: $RE \cap co\text{-}RE = R$.



The language class RE lines up with what we understand to be **computable**.

The Chomsky Hierarchy is just a start. There are hundreds (no kidding) of other classes of languages. Perhaps the two most famous are:

- P: The set of all languages that can be recognized by a deterministic TM in a number of steps bounded by a fixed polynomial in the size of the input.
- NP: The set of all languages that can be recognized by a TM in a number of steps bounded by a fixed polynomial in the size of the input.

Note, by definition, $P \subseteq NP$, but no one has been able to prove whether or not $P = NP$. There is a cash prize waiting for *you* if you can do it.

There's so much more, including languages tied to probabilistic and quantum computing. All of the language families described above, as well as hundreds more, can be found in the Complexity Zoo.

> ***Exercise**: Research and provide definitions for these other classes of languages: LL, $LOGSPACE$, $PSPACE$, $EXPSPACE$, $EXPTIME$, $LALR$, SLR, BPP, RP, and $LINEARSIZE$.*

Semantics

Up to now, our only definition of "language" has been a set of strings over some alphabet, and we've looked at many ways to describe a set of strings. A description of the set of strings in the language is called a **syntax**. But you probably have a sense that languages exist to convey meaning. The way meaning is attached to utterances of a language is known as **semantics**.

For instance, we saw the language of simple arithmetic expressions above. We gave the grammar:

$$
\begin{array}{lcl}
exp & \longrightarrow & digit^{+} \\
 & | & \texttt{"-"}\ exp \\
 & | & exp\ (\texttt{"+"}\ |\ \texttt{"-"}\ |\ \texttt{"*"}\ |\ \texttt{"/"}\ |\ \texttt{"\%"}\ |\ \texttt{"**"})\ exp \\
 & | & \texttt{"("}\ exp\ \texttt{")"}
\end{array}
$$

and said the language **WAS** the set of strings generated by this grammar. To give the utterances meaning, we could define a function mapping every utterance to its meaning, which in this case would be a number.

> ***Exercise**: Look up the difference between a numeral and a number and make sure you understand it*

deeply.

It's customary to define the semantics of a language by a set of rules that show how to evaluate syntactic forms of the language. Here's a pretty straightforward definition:

$$\frac{}{[\![n]\!] \Downarrow n} \qquad \frac{e \Downarrow x}{[\![-e]\!] \Downarrow -x} \qquad \frac{e_1 \Downarrow x \quad e_2 \Downarrow y}{[\![e_1 + e_2]\!] \Downarrow x + y} \qquad \frac{e_1 \Downarrow x \quad e_2 \Downarrow y}{[\![e_1 - e_2]\!] \Downarrow x - y}$$

$$\frac{e_1 \Downarrow x \quad e_2 \Downarrow y}{[\![e_1 * e_2]\!] \Downarrow x \times y} \qquad \frac{e_1 \Downarrow x \quad e_2 \Downarrow y}{[\![e_1 \,/\, e_2]\!] \Downarrow x \div y} \qquad \frac{e_1 \Downarrow x \quad e_2 \Downarrow y}{[\![e_1 \,\%\, e_2]\!] \Downarrow x \bmod y}$$

$$\frac{e_1 \Downarrow x \quad e_2 \Downarrow y}{[\![e_1 ** e_2]\!] \Downarrow x^y} \qquad \frac{e \Downarrow n}{[\![(e)]\!] \Downarrow n}$$

How do we deal with semantics in formal language theory? Interestingly, semantics tends to live in a different theory—not in "formal" language theory, but rather in a theory affectionately known as PLT.

Programming Language Theory (PLT)

Formal Language Theory, it seems, is only concerned with defining or determining which strings are in a language and which are not. It is not concerned with assigning meaning to those strings. We have to go beyond what we've learned so far to deal with questions of meaning. Linguists study **semiotics**. Computer Scientists study **programming language theory**, otherwise known as PLT.

In PLT, **programming languages** are, yes, fundamentally sets of strings, but where each of the members of the language is a **program**, representing a computation. The concern of the syntactic aspect of the formal (contrived) languages we've seen so far is really just to provide a foundation for studying computability and complexity, while the concern of programming languages is for humans to express powerful computations. Programming languages feature extremely rich syntaxes and complex semantics, and introduce us to two worlds: the static and the dynamic.

PLT is a significant and very active branch of computer science that deals with everything about programming languages, including their design, classification, implementation, interpretation,

translation, analysis, features, expressiveness, limitations, syntax, semantics, and so on. It is concerned with:

Syntax

How do we formally specify the **structure** of programs?

Semantics

How do we formally specify the **meaning** of programs?

Type Theory

How do we classify constructs and entities in our programs (by their structure and behavior)?

Static Analysis

What can we say about a program just by looking at it (without running it)?

Translation

How do we construct **compilers** (translators) and **interpreters** (executors)?

Runtime Systems

What do programs look like at runtime, and what do we need to execute them?

Verification

Metapro-gramming

How do we mechanically prove program properties, such as correctness and security?

How do we write programs that operate on, and generate, other programs?

Classification

On what bases can we rigorously classify PLs in meaningful ways?

Basically, anything in the study of Programming Languages seems to fall under the topic of PLT. But in the popular culture, the PLT community seems to be associated with the most theoretical, the most mathematical, the most foundational. PLT people write papers whose formulas have a lot of Greek letters. Logic and category theory are everywhere. And when it comes to actual programming, the PLT community is all over the most cutting-edge stuff, like dependent types. And they appear to favor functional programming languages. Haskell, Scheme, and the ML Family of Languages show up a lot in current research. So do proof assistants, like Agda, Coq, and Lean.

This is good, though.

PLT is a huge field but there is good news for students: Some one has already built, and is currently maintaining, a most excellent curated portal for PLT. So many great links to books, papers, articles, tutorials, and even to other portals! Browse and lose yourself in the good stuff.

You can also begin a journey at Wikipedia's PLT article.

Recall Practice

Here are some questions useful for your spaced repetition learning. Many of the answers are not found on this page. Some will have popped up in lecture. Others will require you to do your own research.

Why do we need a theory of language? HIDE ANSWER

A theory of language allows us to rigorously define exactly how we may express and transmit information.

What is information and how do we conventionally represent it? HIDE ANSWER

Information is that which informs. Technically, it can be defined as a resolution of uncertainty. We represent it with a string of symbols.

What is an alphabet, in language theory? HIDE ANSWER

A finite set of symbols.

What is a string, in language theory? HIDE ANSWER

A finite sequence of symbols over a given alphabet.

How do we denote the empty string (the string of zero symbols)? HIDE ANSWER

ε

What is $|abbabcb|$? HIDE ANSWER

7

How do we denote the concatenation of two strings x and y? HIDE ANSWER

xy

What is ho^3? What is $(ho)^3$ HIDE ANSWER

hooo

hohoho

Why is the traditional representation of concatenation and repetition so annoying?

HIDE ANSWER

String concatentation is written like multiplication, but it is really a kind of addition. String repetition is written like exponentiation, but it is really a kind of multiplication.

What are the substrings of abc? HIDE ANSWER

$\varepsilon, a, b, c, ab, bc, abc$.

What is a language, in formal language theory? HIDE ANSWER

A set of strings over some finite alphabet.

What do we call a member of a language? HIDE ANSWER

An utterance.

What is the difference between $\varnothing$ and $\{\varepsilon\}$? HIDE ANSWER

The former is the language of zero strings; the latter is a language with one string, namely the empty string.

How do the Kleene Closure and the Positive Closure of a language differ? HIDE ANSWER

The Kleene Closure includes the empty string; the Positive Closure does not.

How do we denote the concatenation of two languages, L_1 and L_2? HIDE ANSWER

L_1L_2

What is $\{a, b, c\}^2$? HIDE ANSWER

The set of all strings of length 2 over the alphabet $\{a, b, c\}$, namely $\{aa, ab, ac, ba, bb, bc, ca, cb, cc\}$.

For language L, what are $L\varnothing$ and $L\{\varepsilon\}$? HIDE ANSWER

$\varnothing$ and L, respectively.

What are the two means by which we can formally, but "computationally," define a language, other than listing all the strings or providing a description with a set comprehension? HIDE ANSWER

By generation or recognition.

What is the primary mechanism for which languages are described by generation? HIDE ANSWER

The generative grammar.

What are **variables** in a generative grammar? HIDE ANSWER

Symbols that are distinct from those symbols in the language that the grammar is defining, that are intended to be replaced by (strings of) symbols in the process of generating strings in the language.

What do the metacharacters $|$, $*$, $+$, and $?$ mean in a generative grammar? HIDE ANSWER

Alternation (OR), repetition (Zero-or-more), positive repetition (One-or-more), and optional (Zero-or-one), respectively.

How do we denote the language defined by grammar G? HIDE ANSWER

$L(G)$

When taking about grammars, we sometimes see the metacharacters $\longrightarrow$ and $\Longrightarrow$. What do they mean? HIDE ANSWER

The former is used to denote a rule in a grammar; the latter is used to denote a derivation step.

When does a derivation in a generative grammar "stop"? HIDE ANSWER

When there are no more variables.

What is the single rule in the generative grammar defining the language of positive odd binary numerals? HIDE ANSWER

$odd \rightarrow$ `("0" | "1")* "1"`

What language is defined by the following generative grammar?

$s \longrightarrow$ `"a"` s? `"b"`

HIDE ANSWER

$\{a^n b^n \mid n \geq 1\}$

How do you write a grammar for the language of nested and balanced parentheses?

HIDE ANSWER

$s \longrightarrow$ `("("` s? `")")*`

Why was writing the grammar for $\{a^n b^n c^n \mid n \geq 0\}$ conceptually harder than writing the grammar for $\{a^n b^n \mid n \geq 0\}$? HIDE ANSWER

Because the former requires either (1) left hand sides with multiple symbols or (2) the use of lookahead operators.

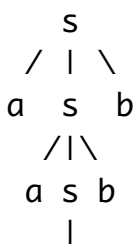
What are some predefined variables that are often used in grammars? HIDE ANSWER

digit, *hexDigit*, *letter*, *alnum*

For the grammar

$s \longrightarrow$ **"a"** $s?$ **"b"**

what does the derivation tree for the string ***aabb*** look like? HIDE ANSWER



The grammar

$s \longrightarrow x \mid y$
$a \longrightarrow x$
$y \longrightarrow b$

defines a language with exactly one utterance, namely ***ab***. But there are two possible derivations. Show them. HIDE ANSWER

- $s \Longrightarrow xy \Longrightarrow ay \Longrightarrow ab$
- $s \Longrightarrow xy \Longrightarrow xb \Longrightarrow ab$

What does it mean for a grammar G to be ambiguous? HIDE ANSWER

There exists a string in $L(G)$ with more than one derivation tree for G.

What are the four components of the mathematical structure representing a formal, generative grammar? HIDE ANSWER

The structure is (V, Σ, R, S) where V is the set of variables, Σ is the alphabet over which the language is defined, R is the set of rules, and S is the start symbol.

In a generative grammar, (V, Σ, R, S), we require $R \subseteq (V \cup \Sigma)^* V (V \cup \Sigma)^* \times (V \cup \Sigma)^*$. Explain this requirement. HIDE ANSWER

Rules map over strings of variables and language symbols (that is, strings from $(V \cup \Sigma)^*$), but the left hand side MUST have at least ONE variable.

Why does the formal definition of a grammar not include ?, $*$, $+$, or $|$? HIDE ANSWER

Because when doing super theoretical meta-level work, brevity and simplicity are key. We want the smallest possible number of building blocks to simplify discussions about what grammars are. We can define these constructs in terms of the basic constructs.

How is the derivation relation $\Longrightarrow_R$ in a generative grammar formally defined? HIDE ANSWER

$\{(xwy, xzy) \mid wRz\}$

What are alternate terms used by different authors for “variable” and language symbol”? HIDE ANSWER

“Nonterminal” and “terminal”

What do we call the union of variables and language symbols? HIDE ANSWER

The vocabulary

What is a context-free grammar (CFG)? HIDE ANSWER

A restricted form of a generative grammar in which the left hand side of every rule is a single variable.

What is a right-linear grammar (RLG)? HIDE ANSWER

A restricted form of a generative grammar in which the right hand side of every rule is a string of zero or more symbols followed by at most one variable.

What is an essentially noncontracting grammar (ENCG)? HIDE ANSWER

A restricted form of a generative grammar in which the right hand side of every rule never has fewer symbols and variables than the left hand side, with one exception: if the language contains the empty string, we can have one rule that maps the start symbol to the empty string.

What are two mechanisms to recognize languages? HIDE ANSWER

Automata and analytic grammars.

What is the most well-known and influential kind of automaton? HIDE ANSWER

The Turing Machine.

What restrictions of Turing Machines correspond to RLGs, CFGs, and ENCGs? HIDE ANSWER

Finite Automata, Pushdown Automata, and Linear Bounded Automata, respectively.

What is the difference between a deterministic and a nondeterministic automaton? HIDE ANSWER

Deterministic automata have only one possible transition to take when in a given configuration; nondeterministic automata have more than one.

What is the difference between a generative and an analytic grammar? HIDE ANSWER

Generative grammars generate all and only the strings in a language. Analytic grammars define a process to determing whether a candidate string is in the language or not.

What is the language defined by the analytic grammar $s \longleftarrow$ `"a"` / `"ab"`? HIDE ANSWER

The language $\{a\}$. (Importantly ab is not in the language because when presented with that string, the grammar matches the a part and will not backtrack to try the second alternative.)

Although analytic grammars have only a single variable on the left hand side of each rule, how can they define non-context-free languages? HIDE ANSWER

By using lookahead operators.

The classic arrangement of language classes in terms of computation power is known as the _____. HIDE ANSWER

The Chomsky Hierarchy

What is a finite language and what kinds of grammars generate them and what kinds of automata recognize them? HIDE ANSWER

A language with only a finite number of strings. They are generated by a grammar in which every rule maps the start symbol to a string in the language. They are recognized by a loop-free finite automaton.

What is a regular language and what kinds of grammars generate them and what kinds of automata recognize them? HIDE ANSWER

A language in which strings can be recognized by processing them one symbol at a time, without backtracking, and without using additional memory. They are generated by a right-linear grammar. They are recognized by a finite automaton.

What is a context-free language and what kinds of grammars generate them and what kinds of automata recognize them? HIDE ANSWER

A language in which strings can be recognized by processing them one symbol at a time, without backtracking, and by using only a simple stack for working memory. They are generated by a CFG. They are recognized by a PDA.

What is a Type 1 language and what kinds of grammars generate them and what kinds of automata recognize them? HIDE ANSWER

A language in which strings can be recognized with fixed apriori bounds on working memory. They are generated by an ENCG. They are recognized by an LBA.

What is the difference between a recursive and a recursively enumerable language? HIDE ANSWER

Recursive languages are decidable, that is, recognizable on a always-halting Turing Machine; recursively enumerable languages are partially decidable, meaning they can be recognized by a Turning Machine that always halts for YES answers but may loop forever on strings not in the language.

Arrange R, FINITE, RE, DCFL, CFL, and REG in subset order. HIDE ANSWER

FINITE $\subset$ REG $\subset$ DCFL $\subset$ CFL $\subset$ R $\subset$ RE

What is $RE \cap$ co-RE? HIDE ANSWER

R

What is a “computable language”? HIDE ANSWER

A language that is recursively enumerable.

What is the language class P? HIDE ANSWER

The set of languages that can be recognized by a deterministic TM in polynomial time.

What is the language class NP? HIDE ANSWER

The set of languages that can be recognized by a TM in polynomial time.

What is semantics concerned with? HIDE ANSWER

The meaning of utterances in a language.

What is the difference between a number and a numeral? HIDE ANSWER

A number is an abstract concept; a numeral is a representation of a number.

What are some of the concerns of PLT? Try to name at least five. HIDE ANSWER

Syntax, Semantics, Type Theory, Static Analysis, Translation, Runtime Systems, Verification, Metaprogramming, Classification.

Who is the curator of the GitHub portal for PLT? HIDE ANSWER

Steven Shaw

Summary

We've covered:

- ✅ Symbols, alphabets, and strings
- ✅ Formal Languages

- ✅ Grammars
- ✅ Parse Trees
- ✅ Ambiguity
- ✅ The formal definition of a grammar
- ✅ Context-Free, Right-Linear, and Type-1 Grammars
- ✅ Language Recognition
- ✅ The Chomsky Hierarchy
- ✅ A bit of history
- ✅ What Programming Language Theory is concerned with

Syntax

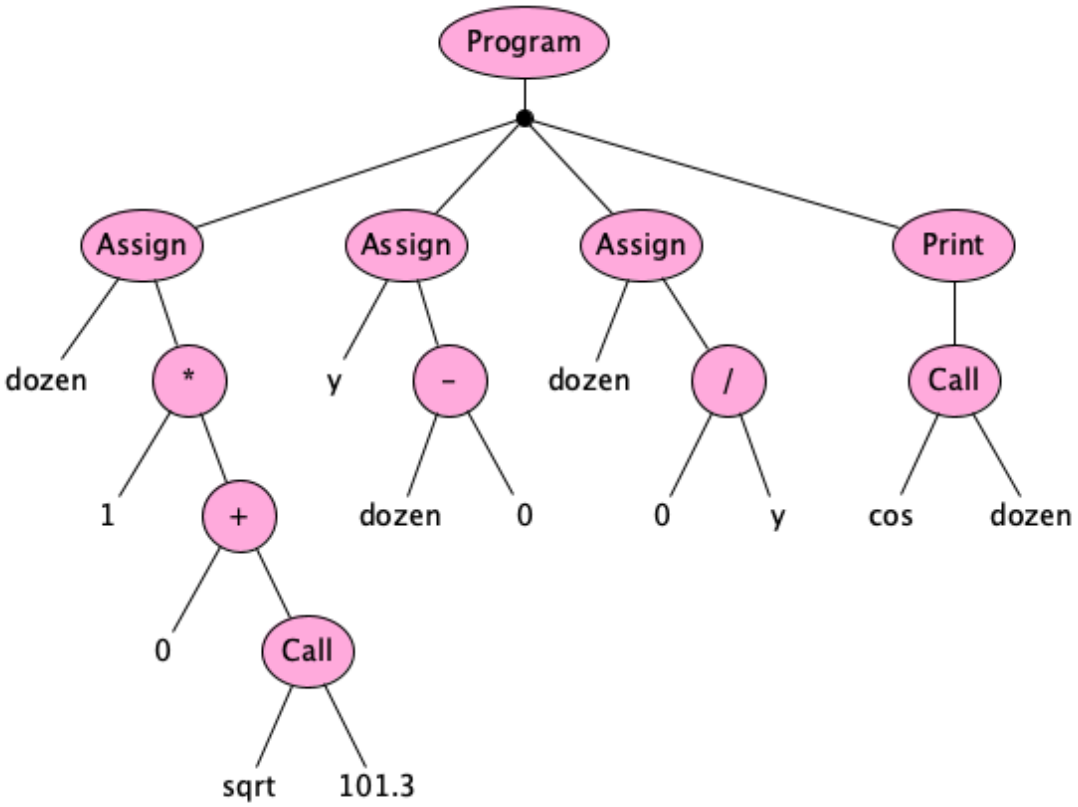
*What is **syntax** and how does it differ, from say, semantics? What are the different ways that we can define a syntax? What is the difference between concrete and abstract syntax? And why do programming languages tend to use context-free grammars for their syntax?*

CONTENTS

Motivation • Definition • Learning by Doing • Lexical and Phrase Syntax • Dealing With Ambiguity • Grammars for Programming Languages • The Problem of Context • Abstract Syntax • Syntax in the Real World • Alternate Syntactic Descriptions • Recall Practice • Summary

Motivation

A language gives us a way structure our thoughts. Utterances in a language (called **programs** in a programming language) have a kind of internal structure, for example:



We generally capture this structure as a sequence of symbols; for example, the structure above might appear as:

```
dozen = 1 * (0 + sqrt(101.3));
y = dozen - 0;    // TADA 🥑
dozen = 0 / y;
print cos(dozen);//
```

or even:

```
MOVE PRODUCT OF 1 AND (SUM OF 0 AND SQRT OF 101.3) INTO DOZEN.
MOVE DIFFERENCE OF DOZEN AND 0 INTO Y.
COMMENT TADA 🥑
MOVE QUOTIENT OF 0 AND Y INTO DOZEN.
PRINT COSINE OF DOZEN.
COMMENT
```

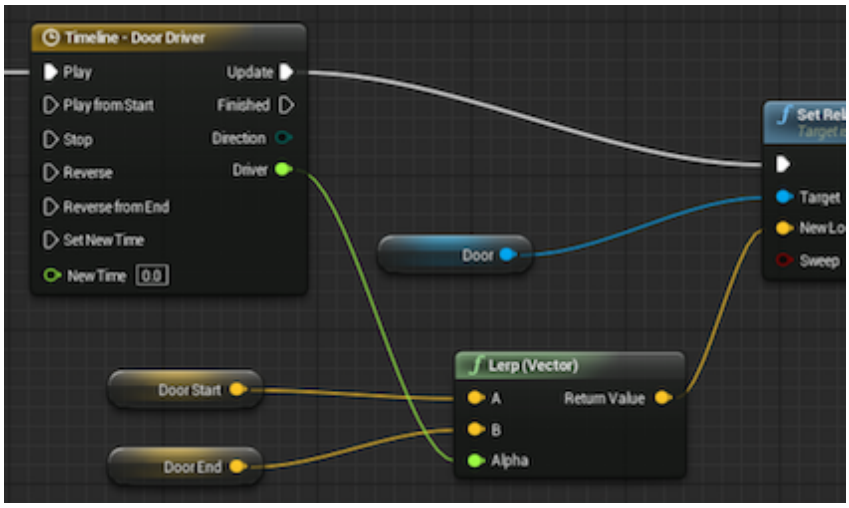
or even:

```
(program
  (assign dozen (* 1 (+ 0 (call sqrt 101.3))))
```

```
(assign y (- dozen 0))          ; TADA 🥑
(assign dozen (/ 0 y))
(print (call cos dozen))        ;
```

Of course, we don't have to use strings! Visual languages do exist:





> ***Text vs. Pictures***
>
> *Text is more dense than pictures, and more directly machine readable at times (no computer vision machine learning necessary). Even if your language was expressed with pictures, there will be an underlying byte-array, or string, representation for storage and transmission.*
>
> *So our focus from now on will be on text.*

***Exercise**: Text is by no means always better than pictures! Please watch Bret Victor's [The Future of Programming] for thoughts on programming visually.*

If you were a language designer, you'd deal with questions such as:

- Is code expressed in terms of blocks? Visual graphs? Text?
- If text, which characters are allowed? Is whitespace significant? Is case significant?
- Is there a lot of punctuation? Symbols? Or is it mostly words? If words, what natural language are you biased toward?
- How do we do comments? Pragmas? Directives?
- Do we express structure with indentation, curly-braces, terminating-end, or nested parentheses?
- Should operations have a functional (`length(a)`) or message-oriented (`a.length`) feel?
- What kind of shorthands and idioms should be made available?
- How do program units map to files or other physical containers?

Definition

The **syntax** of a programming language is the set of rules that define which arrangements of symbols comprise structurally legal programs—and for each structurally legal program, what its structure actually is.

Syntactic rules should be able to tell us that these sentences are not well-formed in English:

- `telescope a they the With hit saw.`
- `yqvfe.,--)AO7%;;;;;;;; ; !dogs`

but that these are:

- `They hit the telescope with a saw.`
- `They saw the hit with a telescope.`

and so is this famous one, due to Chomsky:

- `Colorless green ideas sleep furiously.`

Learning by Doing

Let's learn about syntax by going through the process of designing a syntax for a small language.

We'll assume the preliminary phases of language design—identifying the purpose (education), scope (simple arithmetic computation), audience (students), and name (Astro) of our language—have been done, and now it is time for the technical work to begin.

We first sketch a program to illustrate the syntactic features we want:

```
// A simple program in Astro

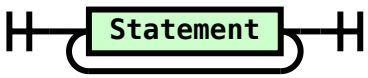
radius = 55.2 * (-cos(2.8E-20) + 89) % 21;    // assignment statement
the_area = π * radius ** 2;                   // a built-in identifier
print hypot(2.28, 3 - radius) / the_area;     // print statement
```

Next, we put our ideas into words. A first pass: "Programs are structured as a sequence of one or more statements, each of which is an assignment or print statement, with expressions formed with numbers, variables, function calls, and the usual arithmetic operators, which can have parentheses when needed. Comments look like the slash-slash comments of C-like languages."

Natural language isn't super precise, so as we sometimes do in design, we'll next turn thoughts into pictures.

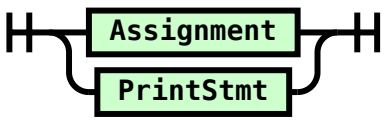
A program is a sequence of one or more statements:

Program



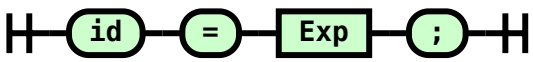
There will be two kinds of statements: assignments and print statements:

Statement

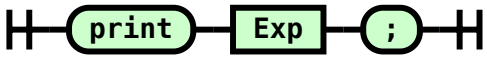


Assignments and print statements look like this:

Assignment



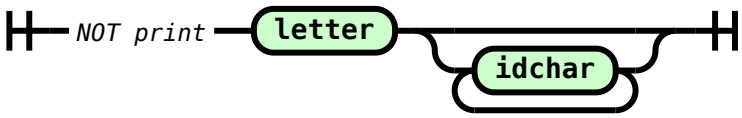
PrintStmt



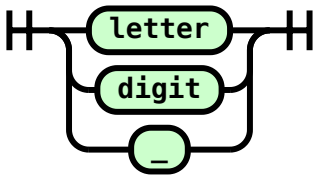
An **identifier** is the computer science word for a *name* you attach to an **entity** (a variable, constant, function, type, parameter, or similar thing). A **keyword** is a special word in your language, like `let` or `if` or `while` —these words generally do not name things—they are just used to structure the code. If a keyword is not allowed to be an identifier, we call it a **reserved word**. Astro has one reserved word: `print`.

Identifiers in Astro begin with a letter, and can have letters, digits, and underscores (examples: `x`, `last_attempt`, `p1`, `p2`, `overTheLimit`, `bot8675309_jEnNy`). We will call letters, digits, and underscores **identifier characters** (`idchar`s). But notice that `print` is not allowed to be an identifier:

id



idchar



How is the reserved word `print` specified? We have to be careful. It's not just the five letters p, r, i, n, and t. If it were then the program:

```
printy;
```

would be legal! It would be the five characters spelling print followed by a legal expression, namely the identifer ***y***. We want the word `print` to not bleed into any following characters that might be part of an expression. In other words, `print` must not be immediately followed by an identifier character:

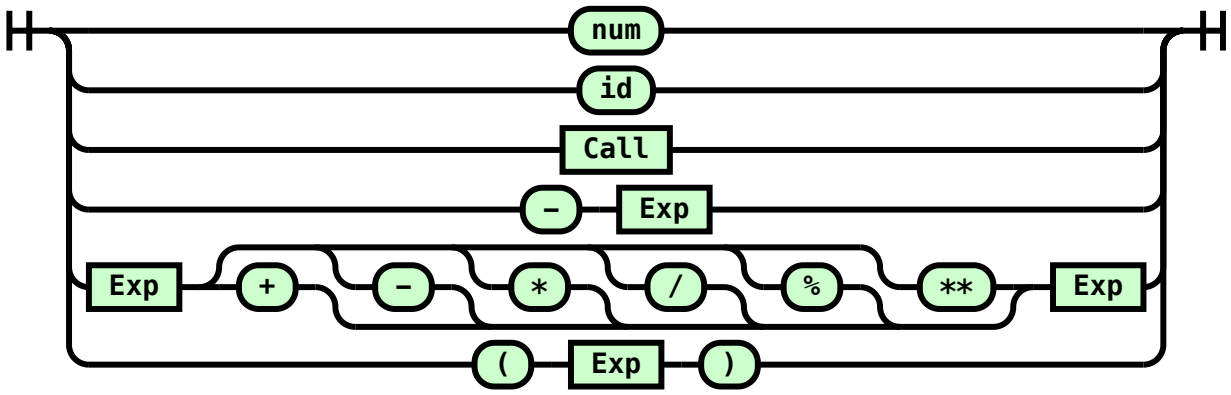
print



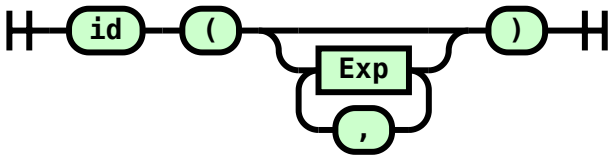
***Exercise**: Make sure you understand how this definition ensures that "printy" is an identifier and not* `print` *followed by the identifier* `y`.

A **statement** performs an action (perhaps with a side effect), but **expressions** represent the computation of a value. Expressions can be numbers, identifiers (representing variables), function calls, or can be made up of operators applied to other expressions. We can put parentheses around any expression. This will do the trick (but spoiler alert: we will have to change it later):

`Exp`



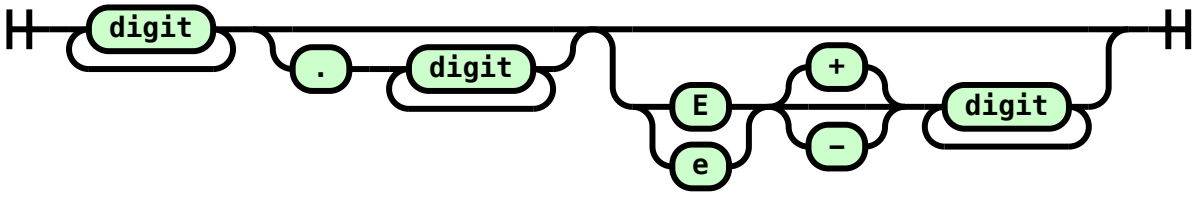
`Call`



***Exercise**: Explain the difference between expressions and statements to a friend. Make yourself a sacred vow never to mix the terms up. Precise language is a good thing. Sometimes we slip up, but still, let's try not to say things like "plus statement."*

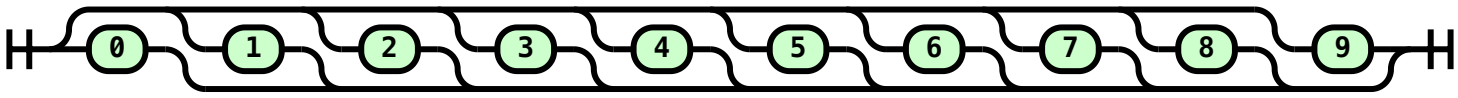
We mentioned numerals here, so let's define these. We'll keep things in decimal only (no worries about hex or binary), and use the times-ten-to-the notation from popular programming languages:

`num`



What about letters and digits? Well digits are pretty easy:

digit

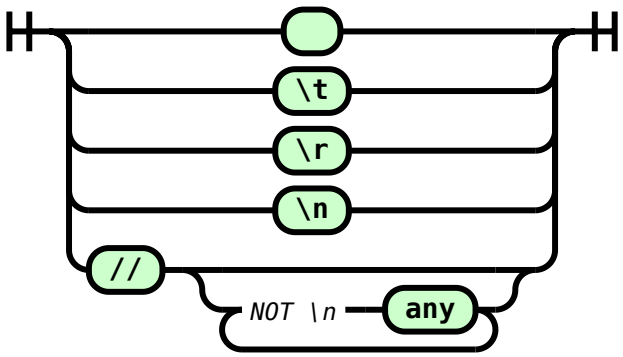


For letters, well, there are tens of thousands of these in Unicode, so we'll just say the rule for `letter` is "built-in."

Lexical and Phrase Syntax

Did you notice that we used two kinds of nodes in our syntax diagrams? We used ovals for constructs that internally cannot have spaces in them; we call these **lexical nodes**, or **tokens**. We used rectangles for constructs whose components can have spaces between them; we call these **phrase nodes**, or **phrases**. The special category called `space` defines what can separate tokens within phrases.

space



In the little language Astro that we are designing, the separators are the space character, the tab character, the carriage return character, the linefeed character (the linefeed (U+000A) is pretty much universally accepted as the "newline"), and comments. Comments function as spaces, too! For comments, we chose to start with `//` and go to the end of the current line.

We also followed an accepted convention to start the names of lexical nodes with lowercase letters and capitalize the names of phrase nodes.

Another convention we will follow for programming language syntax is to take the following as predefined:

- `letter`, for any Unicode letter

- `digit`, for `"0".."9"`
- `alnum`, for `letter | digit`
- `upper`, for any Unicode uppercase letter
- `lower`, for any Unicode lowercase letter
- `hexDigit`, for `digit | "a".."f" | "A".."F"`
- `any`, for any Unicode character at all

and also allow these conveniences:

- `\u{` and `}`, delimit a code point in hex and stands for the unicode character with that code point
- `\n`, for the LF character (same as `\u{a}`)
- `\r`, for the CR character (same as `\u{d}`)
- `\t`, for the tab character (same as `\u{9}`)
- `\"`, for the quote character (same as `\u{22}`)
- `\'`, for the apostrophe character (same as `\u{27}`)
- `\\`, for the backslash character itself (same as `\u{5c}`)

Note that `any` can be defined as `\u{0}..\u{10ffff}`.

> ***Remember***
>
> *The term* `space`*, though it looks like a lexical node, is something meta; it is treated specially.*

Let's spend some more time on the philosophical significance of the separation of lexical and phrase syntax.

The lexical syntax, with the exception of the special rule for `space`, show us how to combine characters into words and punctuation which we'll call **tokens**. The phrase syntax show us how to combine words and symbols into **phrases** (where the words and symbols can, if you wish, be surrounded by spacing).

> ***Exercise**: Meditate on how profound this is. Have you seen anything like this before? Hint: Are you reading this page in English?*

Now if we wanted to **recognize** whether a string is "in" a language, we can figure this out in two phases. The first phases uses the lexical rules to **tokenize** a stream of characters into a stream of

tokens, and the second phase uses the phrase rules to **parse** the token stream into the **parse tree**. Here's an example. The source string:

```
  print( // 麻🌿
420 );
```

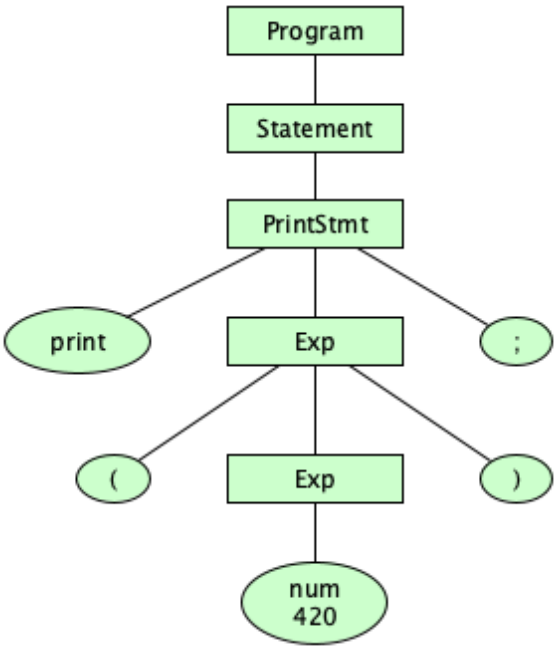
is made up of these characters:

SPACE *SPACE* *LATIN SMALL LETTER P* *LATIN SMALL LETTER R* *LATIN SMALL LETTER I* *LATIN SMALL LETTER N* *LATIN SMALL LETTER T* *LEFT PARENTHESIS* *TAB* *SOLIDUS* *SOLIDUS* *SPACE* *KANGXI RADICAL HEMP* *HERB* *LINEFEED* *DIGIT FOUR* *DIGIT TWO* *DIGIT ZERO* *SPACE* *RIGHT PARENTHESIS* *SEMICOLON*

When we follow the lexical rules to this character stream, skipping the spaces (and comments), we get the token stream:

`print` `(` `num(420)` `)` `;`

Next, we follow the phrase rules allows us to uncover the parse tree:



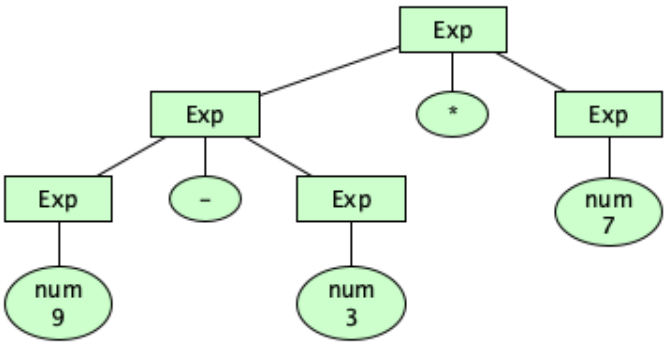
A very important thing to note: **The frontier of the parse tree is the token stream.**

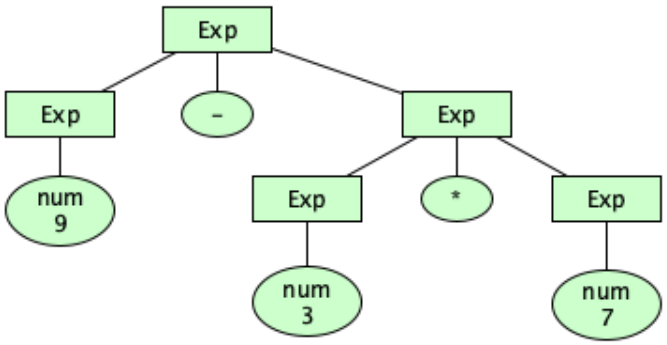
> ***Exercise**: Repeat this phrase to yourself five times: The frontier of the parse tree is the token stream.*

> ***The parse tree ends at tokens, not characters***
>
> *Please take the time to consider how silly it would be if the parse tree expanded all of the lexical rules down to characters. Having the parse tree stop at tokens is a good example of what we would call "breaking a complex problem down into simpler parts."*

Dealing With Ambiguity

How are we doing so far? We are able to distinguish well-structured Astro programs from all other Unicode strings. But there are some things we haven't dealt with yet. For one, we have some strings with multiple structural forms. For example, the phrase `9-3*7` can be **parsed** in two ways:





Having more than one parse tree for a given input string means that our syntax description is **ambiguous**. It's possible to handle this particular kind of ambiguity in the syntax. Here's how.

Precedence

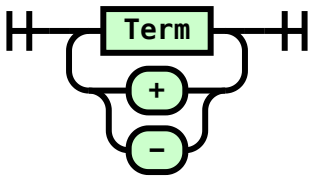
We can create rules that force certain operators to be applied before others; that is, the **precedence** of operators can be enforced in our syntax definition. To do so, we define additional syntactic categories. We say:

- An expression is a sequence of one or more **terms**, separated by additive operators.
- A term is a sequence of one or more **factors**, separated by multiplicative operators.
- A factor is made up of **primaries**, separated by exponentiation operators, or a factor can just be a negated primary.

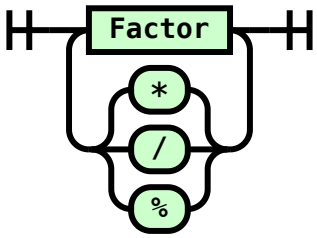
- A primary is the most basic expression possible: either a simple identfier, a numeral, a call, or a parenthesized expression. Parentheses allow unlimited nesting.

We can write:

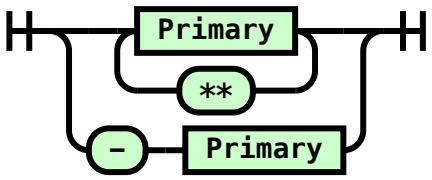
Exp



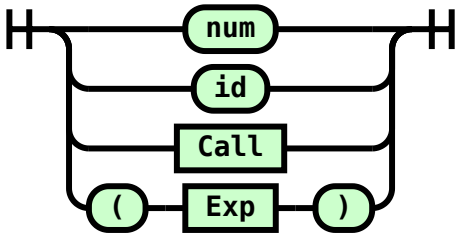
Term



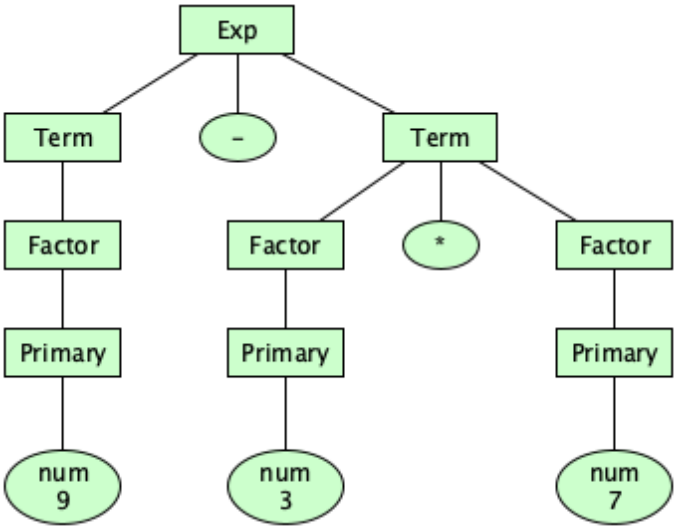
Factor



Primary



Great! Now there is one and only one parse tree for that previously problematic expression:



Note that the new syntax has forced the binary operators into a precedence hierarchy!

- `+` and `-` have the lowest precedence.
- `*`, `/`, and `/` have the next higher precedence.
- `**` has the highest precedence.

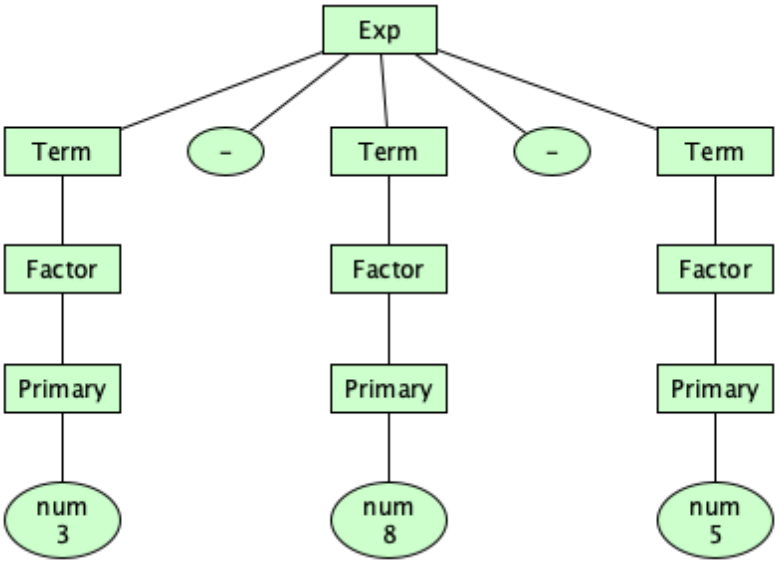
Of course, you can think of parenthesized expressions as being done before anything else, though we don't usually think of these as operators.

> ***Exercise**: Precedence is usually a concept that applies only to binary operators, but they way in which unary operators mix with binary operators does need to be addressed. For instance, how should `-3**2` be parsed? In Python exponentiation precedes negation, like `-(3**2)`. In Elm, negation precedes exponentiation, like `(-3)**2`. Astro follows JavaScript and simply does not allow this expression (forcing programmers to use parentheses in this case)! Show how this is done.*

> ***Exercise**: Our syntax diagram above does not allow `-3**2` as an expression, but it does allow `-3+2` and `-3*2`. Why did we care only enough to ensure negation did not mix with exponentiation, but we were fine with it mixing with addition and multiplication?*

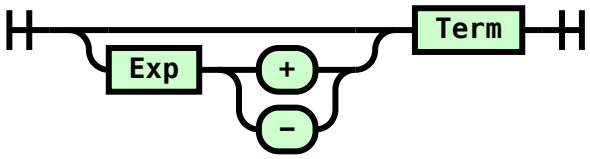
Associativity

Wait, we are not done with structuring our operators just yet. The way things stand now, the parse tree for `3-8-5` looks pretty flat:

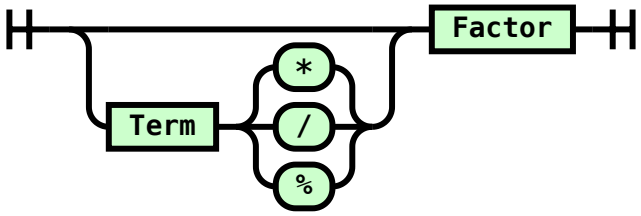


It doesn't suggest whether we mean to compute `(3-8)-5` (which would be -10) or `3-(8-5)` (which would be 0). We can give a syntax that makes this clear. In our design, let's make the additive and multiplicative operators **left-associative** and the exponentiation operator **right-associative**:

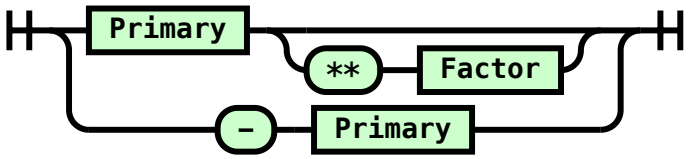
`Exp`



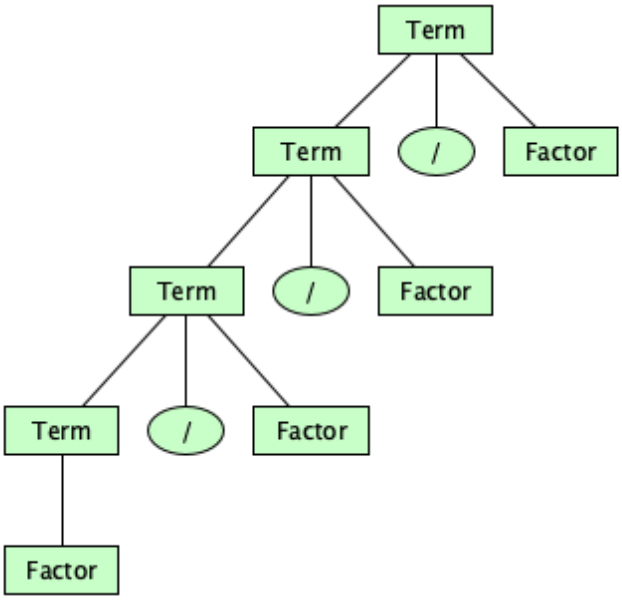
`Term`

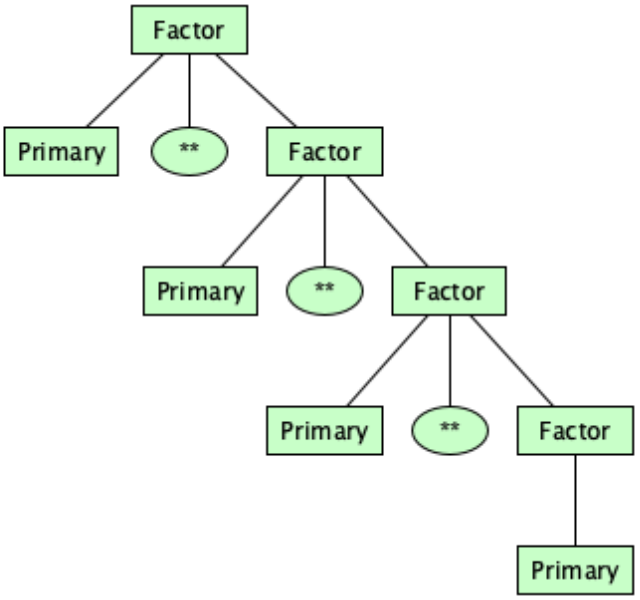


`Factor`



How the heck does this work? Study these parse trees, and hopefully the insight will come to you! (Hint: remember the syntax is *designed* to force the tree to "come out" a certain way.





***Exercise**: Study this until you understand it well.*

Grammars for Programming Languages

Syntax Diagrams are fine for giving an overview of program structure, but, like all pictorial notations, they can take up a lot of space and are rather hard to process by a machine. If we could write a denser, text-based description of a syntax, we could write a program that checked a candidate program for conformance with the syntax! This is exactly what compilers, interpreters, and related tools do.

Let's translate the syntax diagrams above into a grammar for Astro. We will use an analytic grammar, rather than a generative one, as we want to take advantage of negative lookahead to help describe our keywords and identifiers.

Program	⟵	*Statement*$^+$
Statement	⟵	*Assignment* / *PrintStmt*
Assignment	⟵	*id* "=" *Exp* ";"
PrintStmt	⟵	*print Exp* ";"
Exp	⟵	(*Exp* ("+"/"-"))? *Term*
Term	⟵	(*Term* ("*"/"/"/"%"))? *Factor*
Factor	⟵	*Primary* ("**" *Factor*)? / "-" *Primary*
Primary	⟵	*Call* / *id* / *num* / "(" *Exp* ")"
Call	⟵	*id* "(" (*Exp* ("," *Exp*)*)? ")"
numeral	⟵	*digit*$^+$ ("." *digit*$^+$)? (("E"/"e") ("+"/"−")? *digit*$^+$)?
id	⟵	∼*print letter idchar**
idchar	⟵	*letter* / *digit* / "_"
print	⟵	"print" ∼*idchar*
space	⟵	" " / "\t" / "\r" / "\n" / "//" (∼"\n" *any*)*

***Exercise**: Because we used an analytic grammar, it was imperative that we put the Call choice before the id choice in the Primary rule. What would happen if we reversed the order of these two choices?*

***Exercise**: Compare, in detail, the grammar for Astro with the syntax diagrams we've been using. Do you see how they are related?*

The Problem of Context

Question: Does the grammar we have defined above for Astro capture the following (desirable) rule:

> "You can only use an identifier if it has been previously assigned to."

Answer: It does not.

***Exercise**: Show that it does not. Hint: Is `print x;` a legal program according to the grammar?*

***Exercise**: Show how to capture the rule in a grammar. Hint: Do not be frustrated if you cannot.*

Enforcing this rule requires knowledge of **context**. That is, the syntactic rule for producing expressions such as calls and arithmetic operations would need to somehow know which identifiers appeared on the left hand side of some previous assignment statement. This turns out to be so hard

that even designers of real programming languages omit enforcement of contextual rules from the grammar! In fact, while the official grammar for Java will not derive this program:

```
class A {2 / == {{;
```

and report a **syntax error**, a Java compiler will say that this program:

```
class A {int x = y;}
```

is structurally well formed according to the official syntax of the Java language! The compilation unit consists of a type declaration that is a class declaration whose body consists of a field declaration with a type, a name, and an initializing expression which is an identifier. But we know this program is not legal, since the identifier *y* has not been declared. It's not only Java that has grammars overspecifying things: this C program has multiple problems despite being well-formed according to the C grammar:

```
/*
 * This program is derivable by the official grammar for C.
 */
int main() {
    f(x)->p = sqrt(3.3);
    return "dog";
}
```

This is because it's really really really really really really really really hard and really really really really really really really really ugly to define contextual rules grammatically! Capturing an "all identifiers must be declared before use" constraint would require that a grammar rule for expressions not just generate *any* identifier, but only certain identifiers known from some outside context. Context! In general, capturing context in a grammar leads to grammars that:

- are incredibly difficult for humans to figure out,
- are nearly impossible for humans to read (they're ridiculously long and incredibly weird),
- look nothing at all like the way we would check these rules in a compiler,
- give rise to trees that are weird since expansion of nodes is based on sibling nodes, not just the parent, which makes semantics hard to define, and
- horribly expensive even for a computer to parse.

Try the following exercise if you're not convinced:

***Exercise**: Write grammars for:*

- $\{a^n b^n c^n \mid n \geq 0\}$
- $\{a^{2^n} \mid n \geq 0\}$
- $\{ww \mid w \in \{a, b\}*\}$

Hint 1: Grammars for these languages will have to have at least some rules with multiple symbols on the left hand side. That is because there exist proofs (not shown here) that these three languages are not context-free.

Hint 2: Having trouble? Answers can be found on my notes on Language Theory.

***Exercise**: Do some research to see if there are any known lower complexity bounds on the parsing of context-sensitive grammars.*

Sometimes there are constraints that we *can* (theoretically) capture in a context-free syntax, but that we might not want to. Try the following two exercises:

***Exercise**: Write a syntax rule for nonnegative integers less than* 128.

***Exercise**: Write a syntax rule for nonnegative integers less than* 2^{63}. *Is this busy work?*

Defining Contextual Constraints

So many programming language rules require context. Here's a small sampling from various languages:

- Identifiers must not be redeclared within a scope
- Identifiers may not be used in a place where they might not have been initialized
- Types of all subexpressions must be consistent in an expression
- The correct number of arguments must appear in a call
- Access modifiers (private, protected, public) must be honored
- All execution paths through a function must end in a return
- All possible arms of a match or switch or discriminated union must be covered
- All abstract methods must be implemented or declared abstract
- All declared local variables must be used
- All private methods in a class must be used

- ...and so many more

For our running example language, Astro, our constraints are:

1. The following identifiers are built-in:
    - `π`, a number
    - `sqrt`, a function of exactly one argument
    - `sin`, a function of exactly one argument
    - `cos`, a function of exactly one argument
    - `hypot`, a function of exactly two arguments
2. An identifier cannot be used in an expression unless it is one of the built-in identifiers or has been previously assigned to.
3. All function calls must accept the proper number of arguments.
4. The built-in identifiers cannot be assigned to.
5. Identifiers declared as functions can only be called, not used in any other context. Identifiers declared as numbers cannot be called.

That's informal prose. Can we capture the rules in a formal notations? Well, context sensitive grammars are too hard to write, too hard to understand, and too hard to parse, so we need other options. There are at least two:

- Use a completely new kind of grammatical formalism. Some of these do exist, including Attribute Grammars and Two-Level Grammars.

- Push contextual constraints into the world of semantics. Let the syntax be happy with undeclared identifiers appearing in expressions (as long as the expressions are otherwise well formed). With this approach, we say that using undeclared identifiers is "meaningless," you know, a semantic problem. And because the meaning (ahem, semantics) is computed before the program is run, we would call constraint violations **static semantic errors**.

Which take do you prefer? Slonneger and Kurtz take the former approach, writing:

> *The classification of such an error entails some controversy. Language implementers, such as compiler writers, say that such an infraction belongs to the **static semantics** of a language since it involves the meaning of symbols and can be determined statically, which means solely derived from the text of the program. We argue that static errors belong to the syntax, not the semantics, of a language. Consider a program in which we declare a constant:*
>
> ```
> const c = 5;
> ```

In the context of this declaration, the following assignment commands are erroneous for essentially the same reason: It makes no sense to assign an expression value to a constant.

```
5 := 66;
c := 66;
```

The error in the first command can be determined based on the context-free grammar (BNF) of the language, but the second is normally recognized as part of checking the context constraints. Our view is that both errors involve the incorrect formation of symbols in a command—that is, the syntax of the language. The basis of these syntactic restrictions is to avoid commands that are meaningless given the usual model of the language.

Though it may be difficult to draw the line accurately between syntax and semantics, we hold that issues normally dealt with from the static text should be called syntax, and those that involve a program's behavior during execution be called semantics. Therefore we consider syntax to have two components: the context-free syntax defined by a BNF specification and the context-sensitive syntax consisting of context conditions or constraints that legal programs must obey.

Indentation-Sensitive Syntax

In addition to the context issues that exist between a language's syntax and its static semantics, similar issues exist between the lexical syntax and phrase syntax in languages that are indentation-sensitive. Traditionally, lexical analysis proceeds as if the size of whitespace runs don't matter very much. But in languages like Python and Haskell, they do. A lot. We cannot just skip any whitespace between tokens the way we've done so above!

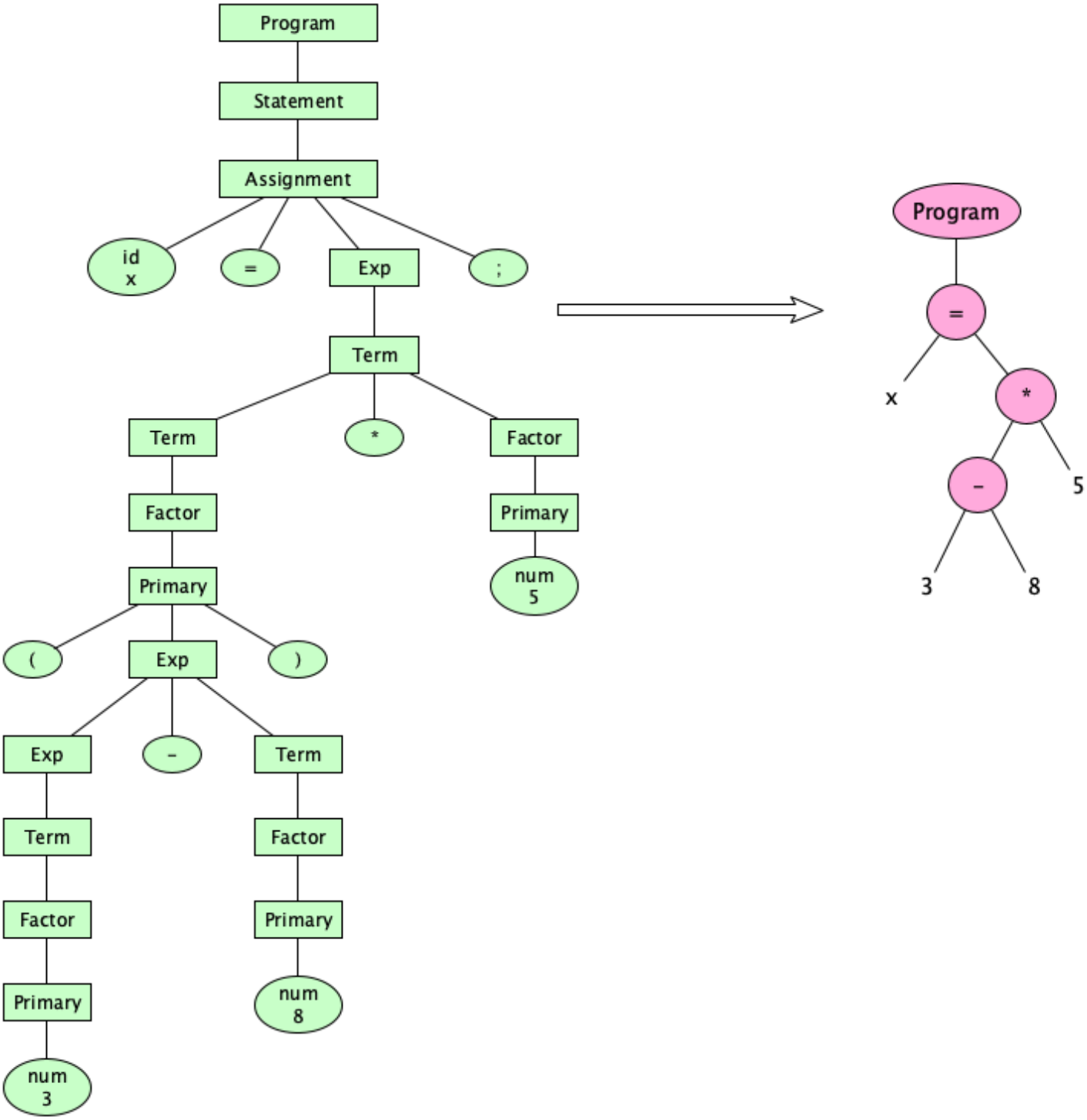
The way to handle indentation sensitivity is, during lexical analysis, keep track of indentation levels (in some sort of context object), and replace certain runs of whitespace with `INDENT` and `DEDENT` tokens.

The process isn't too hard. For complete details, read the section on indentation in the Lexical Analysis chapter of the Python Language Reference.

Abstract Syntax

Parsing is a lot of work. Specifying programs as strings of characters requires extra tokens (parentheses) and extra categories (Term, Factor, and Primary) in order to capture precedence, and crazy rule writing to get associativity correct. We have semicolons to terminate statements. Real languages have lots and lots of punctuation as well. Parse trees can get huge!

Dealing with spaces, comments, tokens, punctuation, precedence, and associativity is important in the early stages of language processing (the thing that interpreters, compilers, syntax highlighters, debuggers, and performance monitoring systems do). But once source code is parsed and the internal structure is uncovered, we can simplify the parse tree into something much simpler for the later stages of language processing (e.g., semantic analysis, optimization, translation, code generation, or execution). These later stages don't really *need* that much from a parse tree. We can abstract away a lot of the messy stuff, and translate our parse tree into an **abstract syntax tree**:



So much simpler! And by the way, since these simpler trees have become known as **abstract syntax trees** (ASTs), it turns out that, yes, parse trees have become known as **concrete syntax trees** (CSTs).

How do we come up with an abstract syntax? Focus on what a program is trying to express, without worrying about punctuation. You don't even have to worry about precedence and associativity (or even parentheses) because that stuff was already "taken care of" earlier. So cool! Here's how we think about it for Astro:

- A program is a sequence of statements
- A statement is either an assignment or a call

- An assignment is an identifier being assigned an expression
- A call has an identifier for the function name and a list of expressions for its arguments
- An expression can be a number, identifier, a unary operator applied to an operand, two expressions with a binary operator, or a call.

If you want to formalize this, you can write a **tree grammar**:

n :	Nml		
i :	Ide		
e :	Exp	$=$	$n \mid i \mid -e \mid e + e \mid e - e \mid e * e \mid e / e \mid e \backslash\% e \mid e ** e \mid i\ e^*$
s :	Stm	$=$	$i = e \mid \texttt{print}\ e$
p :	Pro	$=$	$\texttt{program}\ s^+$

Some people like to write tree grammars another way:

Program → Stmt+
Assign : Stmt → id Exp
Print : Stmt → Exp
num : Exp
id : Exp
Negate : Exp → Exp
Plus : Exp → Exp Exp
Minus : Exp → Exp Exp
Times : Exp → Exp Exp
Divide : Exp → Exp Exp
Remainder : Exp → Exp Exp
Power : Exp → Exp Exp
Call : Exp → id Exp*

When translating to an abstract syntax, we not only get rid of intermediate syntax categories, but we also drop parentheses, commas, semicolons, brackets, and pretty much all punctuation. Some of the smaller keywords that you see in larger programming languages (`in` , `to` , `of`) also are not needed. Punctuation really only exists to give a "tree structure" to our linear program text. Once we have uncovered the tree structure in the CST, we just carry it over naturally to the AST.

Don't break down the tokens either

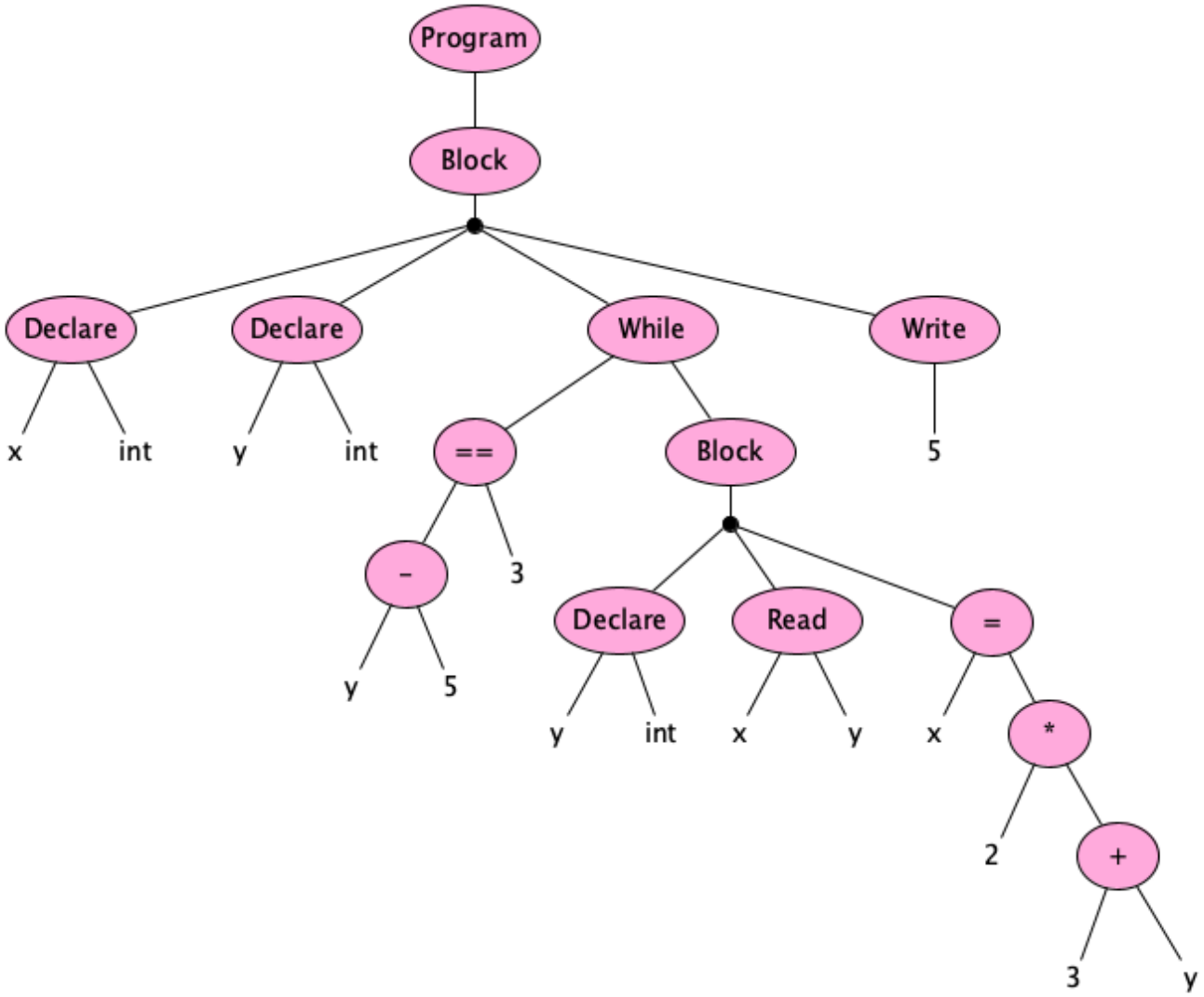
When writing an abstract syntax, you can assume that the tokens (the lexical syntax) is already done for you.All lexical categories (like identifiers and numeric literals in our example above) are primitive.

***Exercise**: Give the token stream and draw the concrete and abstract syntax trees for the Astro program* `print(3-5-8-2*8-9-7**3**1);`.

***Exercise**: Give the token stream and draw the concrete and abstract syntax trees for the Astro expression* `98 * (247 - 6) / 3 / 100 - 701`.

Multiple Concrete Syntax Trees

So ASTs allow us to focus on programming language capabilities without worrying about syntactic details that people can reasonably have different preferences about. Take a look at this AST (in some hypothetical language, not Astro):



CLASSWORK

Stop! Don't scroll by! Study this tree! What is this program "saying"?

It appears to be a program in a language whose tree grammar probably looks like this:

n :	Nml		
i :	Ide		
t :	Type	$=$	**int** | **bool**
e :	Exp	$=$	$n \mid i \mid -e \mid e+e \mid e-e \mid e*e \mid e/e \mid e**e$
			$\mid\ e<e \mid e\leq e \mid e==e \mid e\neq e \mid e>e \mid e\geq e \mid i\ e^*$
s :	Stmt	$=$	$i=e \mid$ **read** $i^+ \mid$ **write** $e^+ \mid$ **while** $e\ b$
d :	Dec	$=$	**declare** $i\ t$
b :	Block	$=$	$d^*\ s^+$
p :	Pro	$=$	**program** b

But what does the actual concrete syntax look like? There's no specific answer. There are an infinite number of possibilities! Here is just a small sampling of programs that could have generated that

AST:

```
program:
    var x, y: int
    // In this language, indentation defines blocks
    while y - 5 == 3:
        var y: int
        get(x)
        get(y)
        x = 2 * (3 + y)
    put(5)
```

```
int x, y;
while y - 5 = 3 {
    int y;
    STDIN -> x;
    STDIN -> y;
    x <- 2 * (3 + y);
}
STDOUT <- 5;
```

```
COMMENT THIS LOOKS LIKE OLD CODE
DECLARE INT X.
DECLARE INT Y.
WHILE DIFFERENCE OF Y AND 5 IS 3 LOOP:
    DECLARE INT Y.
    READ FROM STDIN INTO X.
    READ FROM STDIN INTO Y.
    MOVE PRODUCT OF 2 AND (SUM OF 3 AND Y) INTO X.
END LOOP.
WRITE 5 TO STDOUT.
```

```
(program
  (declare x int)
  (declare y int)
  (while (= (- y 5) 3)
    (define (y int))
    (read x y)
    (assign x (* 2 (+ 3 y)))
  )
```

```
  (write 5)
)
```

```
<program>
  <declare var="x" type="int"/>
  <declare var="y" type="int"/>
  <while>
    <equals>
      <minus><varexp ref="y"/><intlit value="5"/></minus>
      <intlit value="3"/>
    </equals>
    <declare var="y" type="int"/>
    <read vars="x y"/>
    <assign varexp="x">
      <times>
        <intlit value="2"/>
        <plus>
          <intlit value="3"/>
          <varexp ref="y"/>
        </plus>
      </times>
    </assign>
  </while>
  <write>
    <intlit value="5"/>
  </write>
</program>
```

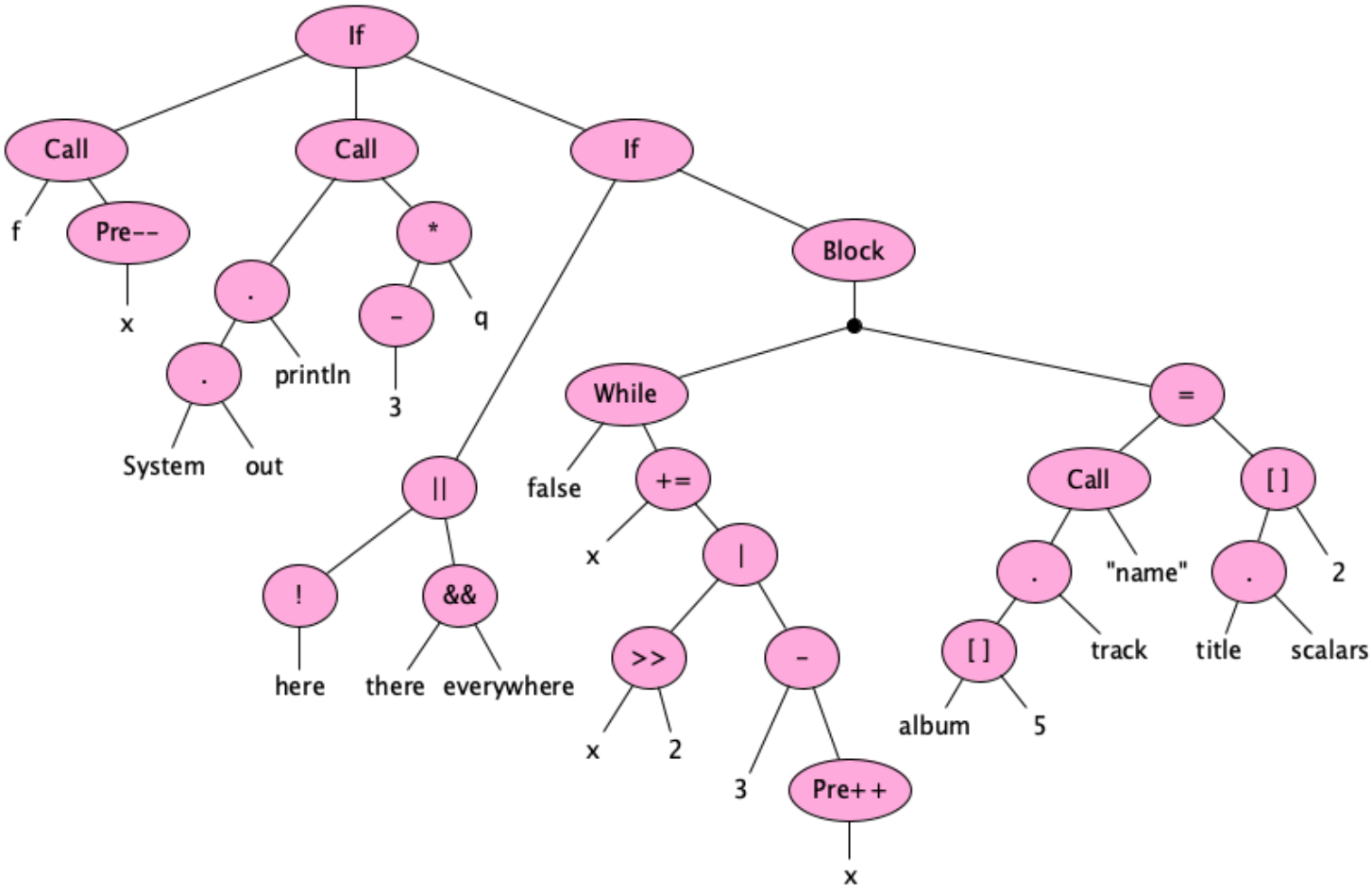
***Exercise**: Write the example in a JSON-like notation.*

***Exercise**: The fourth example above has an "S-expression syntax" which is seen in languages like Lisp and Clojure. In such languages, concrete and abstract syntaxes can be said to be very close to each other. Why is this claim true?*

More AST Examples

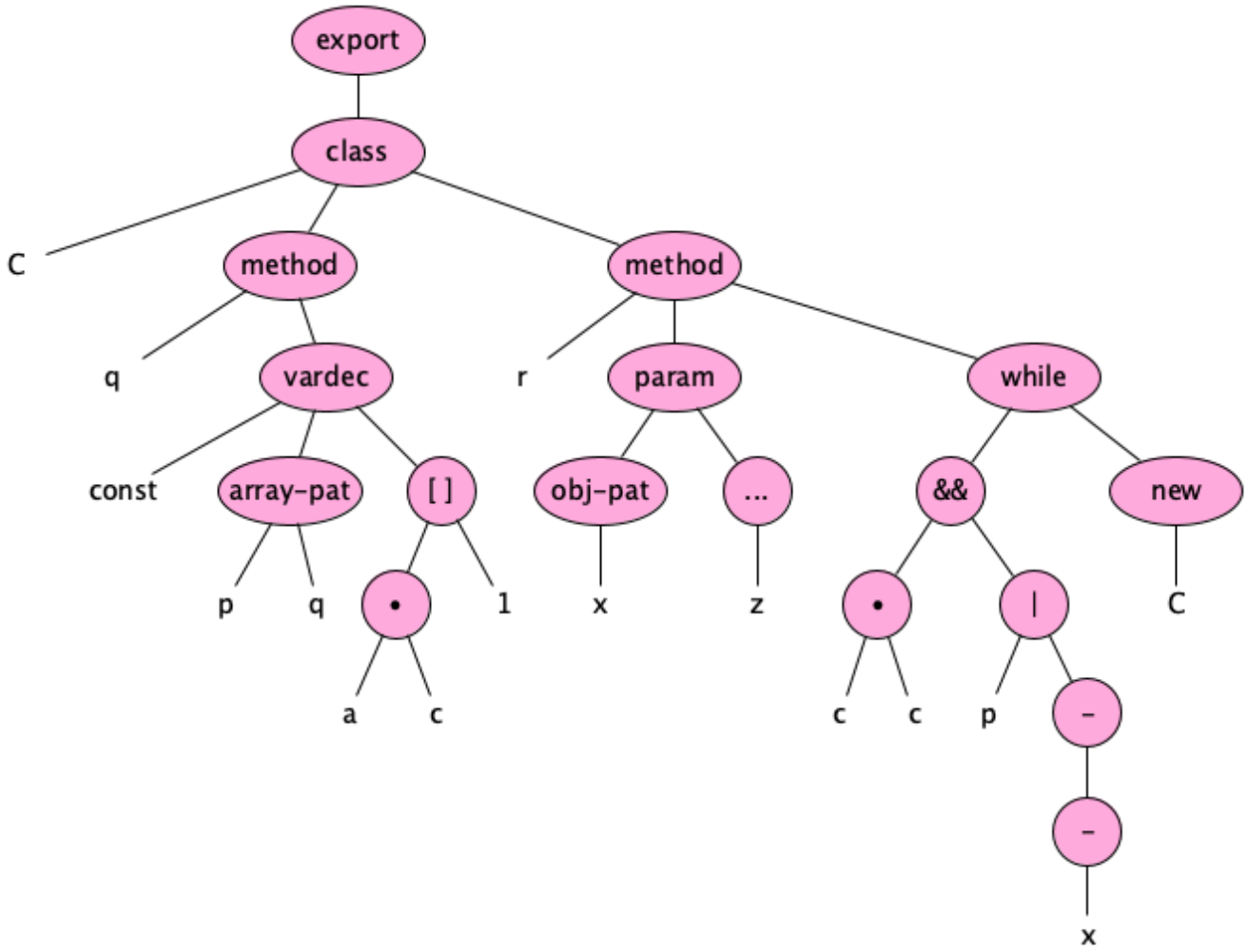
Getting a good feel for abstract syntax naturally takes practice. Here are a couple examples to study to get a feel for how abstract we need to get. Here's a fragment of Java:

```
if (((f(--x)))) {
    System.out.println(- 3*q);
} else if (!here||there    &&everywhere) {
    while (false) {
        x += x >> 2 | 3  -++  x;
    }
    album[5].track("name") = title.scalars[2];
}
```



And here's a little JavaScript program:

```
export class C {
  q() { const [p, q] = a.c[1]}
  r({x}, ...z) { while (c.c&&p|- -x) new C() }
}
```



Exploring Abstract Syntax

I've built an interactive application for you to explore JavaScript abstract syntax trees. Please try it out!. Hopefully you can get a sense of the connection between concrete and abstract syntax by doing so.

Syntax in the Real World

These notes were very much just an introduction to some of the theory surrounding syntax and syntax definitions. There is so much more to study. For example, you would do well to see how much work goes into writing a grammar for a real-world programming language. Here are a few to check out. Note the wide variety of notations:

- Python
- Java

- JavaScript
- Kotlin
- Go
- Swift
- C++
- C

> ***Exercise**: Browse each of these descriptions. Take notes that summarize the basic ideas of each. For each, how to they distinguish lexical from phrase syntax? How to they distinguish tokens from variables? What interesting conventions do they have that you had not thought of?*

> ***Exercise**: Find more official grammars for real languages, and send me links.*

Alternate Syntactic Descriptions

Hey, before we finish up, there's one last thing to cover.

So far, we've seen two approaches to syntax definition: Syntax Diagrams and a form of analytic grammar based strongly on Parsing Expression Grammars. It turns out there are dozens of published notations out there! You can use whatever you like, but there are some standardized forms that you should be aware of. The following four (CFG, BNF, EBNF, ABNF) are **historically significant** and very good to know. (You might even be expected to know these if you have a Computer Science degree.)

> ***These notations may not be as formal as you think!***
>
> *These traditional syntax notations generally do not make distinctions between the lexical and phrase syntaxes. You have to infer by context, or by side notes in a language definition, how tokens are formed and where spaces are allowed to go.*
>
> *This is why you should stick to analytic grammars!*

Context-Free Grammars

Chomsky discovered (or invented?) his Type-2 grammars, which he called Context-Free Grammars, as a minimalistic formalism for studying languages that do not have context. They are by design very "mathematical" and far too low-level to be used in real life. They just use individual symbols. They have no special operators for alternation or repetition. They are too hard to read and are never used directly in practice. A CFG is a 4-tuple (V, Σ, R, S), where V is the set of variables, Σ is the alphabet (of symbols in the language), $V \cap \Sigma = \varnothing$, $R \subseteq V \times (V \cup \Sigma)*$ is the rule set, and $S \in V$ is the start symbol. For the language of simple arithmetic expressions over natural numbers, a CFG is:

$$
\begin{(}
\quad \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, +, -, *, /, (,)\},
\quad \{E, T, F, N, D\},
\quad \{
\qquad (D, 0), (D, 1), (D, 2), (D, 3), (D, 4), (D, 5), (D, 6),
\qquad (D, 7), (D, 8), (D, 9), (N, D), (N, ND), (E, T), (E, E + T),
\qquad (E, E - T), (T, F), (T, T * F), (T, T/F), (F, N), (F, (E))
\quad \},
\quad E
\end{)}
$$

Rather than writing them in tuple form, we often just list the rules, making sure the start symbol appears first, and making the variables and alphabet symbols inferrable by context:

```
E → E + T
E → E - T
E → T
T → T * F
T → T / F
T → F
F → ( E )
F → N
N → N D
N → D
D → 0
D → 1
D → 2
D → 3
D → 4
D → 5
D → 6
D → 7
```

```
D → 8
D → 9
```

A pure CFG is a super-minimal notation great for studying properties of formal, made-up, simple, languages. But when defining the syntax of real languages we want more expressiveness! It's nice to have ways to specify things like keywords and spacing, which get super messy if you stick to the pure CFG notation.

BNF

With BNF, we add the ability to have our variables and alphabet symbols be written in a more readable form. To distinguish the two, we quote the *variables* with angle brackets. One special operator, `|` (meaning "or"), is permitted.

```
<expression> ::= <term>
               |  <expression> + <term>
               |  <expression> - <term>
<term>       ::= <factor>
               |  <term> * <factor>
               |  <term> / <factor>
<factor>     ::= <number>
               |  ( <expression> )
<number>     ::= <digit> | <number> <digit>
<digit>      ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9
```

Better than a CFG, but there's still not built-in way to do keywords and spaces.

EBNF

EBNF, or *Extended* Backus Naur Form, quotes the alphabet symbols instead of the variables, and adds operators for alternation (`|`), optionality (`?`), zero-or-more (`*`), and one-or-more (`+`).

```
EXP    = TERM (('+' | '-') TERM)*
TERM   = FACTOR (('*' | '/') FACTOR)*
FACTOR = NUMBER | '(' EXP ')'
NUMBER = ('0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9')+
```

Now we're getting more concise. Pure EBNF still doesn't have any mechanism for distinguishing lexical and phrase variables, so folks using EBNF will either write two grammars (one for the tokens, and one for the phrases), or use some other convention.

ABNF

ABNF, or *Augmented* Backus Naur Form, is the specification used to describe all the Internet Protocols. It features tools for repetition, alternatives, order-independence, and value ranges.

```
exp    = term *(("+" / "-") term)
term   = factor *(("*" / "/") factor)
factor = number / "(" exp ")"
number = %x30-39
```

Recall Practice

Here are some questions useful for your spaced repetition learning. Many of the answers are not found on this page. Some will have popped up in lecture. Others will require you to do your own research.

What are the utterances of a programming language called? HIDE ANSWER

Programs

What is syntax and how does it differ from semantics? HIDE ANSWER

Syntax is a specification of the structure of a language, while semantics specifies its meaning.

What are some spectra of syntactic forms in common programming languages? HIDE ANSWER

Pictures vs. text

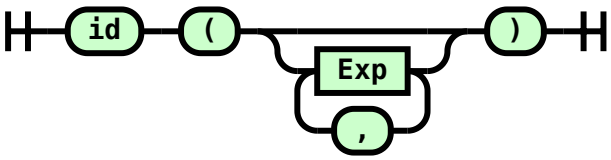
Symbols vs. words

Whitespace significance vs. insignificance

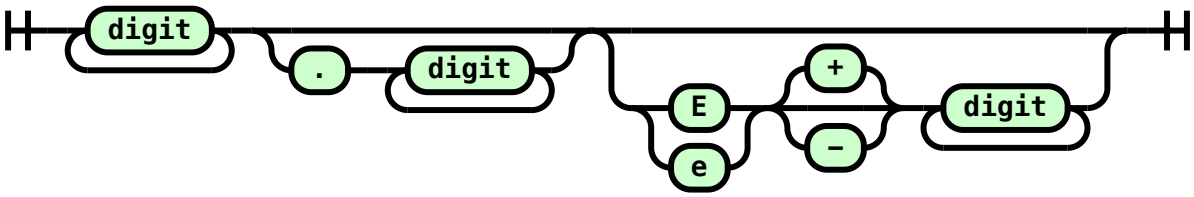
Indentation vs. curly braces vs. ends vs. parentheses

Prefix vs. infix vs. postfix

What does a syntax diagram for a function call look like, assuming the function being called must be a simple identifier? HIDE ANSWER



What does the syntax diagram for a typical floating-point literal look like? HIDE ANSWER



What are entities and identifiers? HIDE ANSWER

An entity is something that is manipulated by a program, such as a variable, literal, function, class, or module. An identifier is a name you can bind to an entity.

What is the difference between a statement and an expression? HIDE ANSWER

An expression is a construct that evaluates to a value, while a statement is a construct that performs an action.

What do we call words that look like identifiers but cannot be used as identifiers? HIDE ANSWER

Reserved words

What is a **reserved word**? HIDE ANSWER

A token that has the same lexical structure as an identifier but is not allowed to be used as one.

How do we, when defining a syntax, make sure reserved words cannot be used as identifiers?

HIDE ANSWER

We start the rule for identifiers with a negative lookahead for reserved words. Also, we need a negative lookahead after each reserved word to make sure it doesn't prohibit identifiers that may have a reserved word as a proper prefix.

What is the difference between the lexical and phrase syntax? HIDE ANSWER

The lexical syntax defines how individual characters are grouped into words, called tokens (such as identifiers, numerals, and punctuation); the phrase syntax groups words into higher level constructs (such as expressions and statements). These words can be separated by whitespace.

Why do grammars make a distinction between lexical and phrase syntaxes? Give two reasons.

HIDE ANSWER

1. To avoid littering nearly every rule with whitespace sequences.
2. To conceptually simplify the syntax, breaking it down into two manageable parts.

What convention are we using on this page to distinguish lexical and phrase variables?

HIDE ANSWER

Lexical variables begin with a lowercase letter; phrase categories begin with an uppercase letter.

What exactly is a **token**? HIDE ANSWER

A primitive element for a grammar's phrase structure, e.g., a keyword, identifier, numeric literal, boolean literal, simple string literal (without interpolation), or symbol.

What happens during **tokenization**? What kinds of language constructs are "eliminated" in this process? HIDE ANSWER

Characters in the source code are grouped into tokens. Comments and spaces between tokens are dropped. Spaces within string literals are kept of course.

Where do we generally cut off a parse tree? HIDE ANSWER

At the token level.

What does it mean for a grammar to be **ambiguous**? Answer in precise, technical terms. HIDE ANSWER

A grammar is ambiguous if there exists a string that has two distinct derivation trees in the grammar.

What kind of rule is a major source of ambiguity, one that is common for beginners to write? HIDE ANSWER

Rules with multiple components the same as the category being defined, e.g., $exp \longrightarrow exp\ op\ exp \mid \cdots$

How is operator precedence captured in a language definition? HIDE ANSWER

We use multiple syntactic categories for expressions; for example Expression, Term, Factor, Primary.

How do we capture the conventional precedences and associativities of binary arithmetic operators in an analytic grammar? HIDE ANSWER

$$
\begin{array}{lcl}
Exp & \longleftarrow & (Exp\ addop)?\ Term \\
Term & \longleftarrow & (Term\ mulop)?\ Factor \\
Factor & \longleftarrow & Primary\ (\texttt{"**"}\ Factor)?
\end{array}
$$

What is a good way to treat the expression `-3**2` in a language definition? HIDE ANSWER

It should be made illegal by the grammar! This eliminates the possible confusion about whether it means `(-3)**2` or `-(3**2)`, since it requires programmers to use parentheses.

What kinds of rules about program legality are not context free? Find as many as you can.

HIDE ANSWER

Some of them are:

- Type checking
- Redeclaration of identifiers within a scope
- Use of an undeclared identifier
- Matching arguments to parameters in a call
- Access checks (public, private, etc.)
- Identifiers must be used in all paths through their scope
- A return must appear in all paths through a function
- Pattern match exhaustiveness
- In a subclass, abstract methods must be implemented or declared abstract
- All private functions must be called within a module

There may be more.

What are two formalisms that have been created to capture context-sensitive rules?

HIDE ANSWER

Attribute Grammars and Two-Level Grammars

What do many language designers do in order to avoid the complexity of context-sensitive grammar rules? HIDE ANSWER

They push the rules into the world of semantics.

What kinds of rules about program legality can in principle be captured in a context-free syntax, but are generally not so captured in practice because the the required complexity of the rules?

HIDE ANSWER

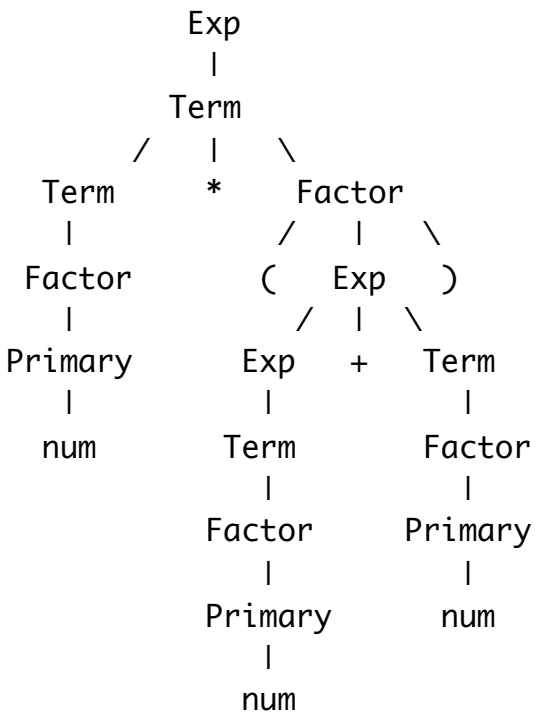
- Requiring `break` and `continue` in a loop (as this would require difference classes of statements)
- Requiring `return` in a function body (as this would require difference classes of statements)
- Enforcing ranges on certain numeric types.

In principle, can you capture the notion at all integer literals (in a hypothetical language) must be between -2147483648 and 2147483647? If so, why don't we? HIDE ANSWER

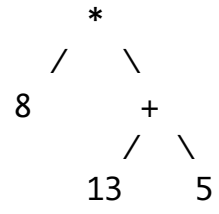
Yes, you certainly can, but the resulting specification is ugly af and really hard to decipher and understand.

Draw the parse tree and the abstract syntax tree for the expression `8 * (13 + 5)`, assuming a grammar with categories named Exp, Term, Factor, Primary, and num. HIDE ANSWER

The parse tree is:



The abstract syntax tree is:



Is indentation-sensitivity context-free? HIDE ANSWER

No

What is a concrete syntax? HIDE ANSWER

A precise specification of which strings (sequences of characters) are structurally legal programs.

What kinds of constructs appear in a concrete syntax but not in an abstract syntax? See how many items you can find. HIDE ANSWER

- Punctuation and delimiters
- Intermediate expression categories, e.g. Exp1, Exp2, Term, Factor, ...
- INDENT and DEDENT tokens

Given the expression `8 * (13  + 5)`, how many source code characters are there? When tokenized, how many tokens are there? How many nodes are there in the abstract syntax tree? HIDE ANSWER

14 characters, 7 tokens, 5 nodes.

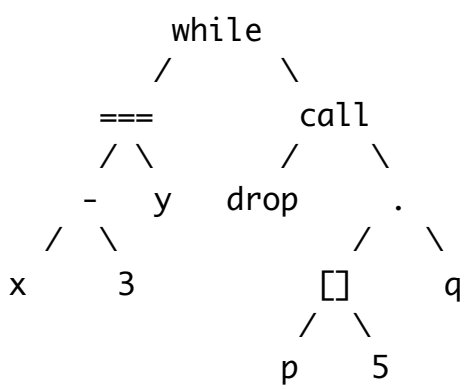
What kind of formal notation can capture an abstract syntax? HIDE ANSWER

A tree grammar.

What does the tree grammar for Astro look like? HIDE ANSWER

n : **Nml**
i : **Ide**
e : **Exp** $=$ $n \mid i \mid -e \mid e + e \mid e - e \mid e * e \mid e / e \mid e ** e \mid i\ e^*$
s : **Stm** $=$ $i = e \mid \mathbf{print}\ e$
p : **Pro** $=$ $\mathbf{program}\ s^+$

What does the AST for the JavaScript expression `while (x - 3 === y) { drop(p[5].q) }` look like? HIDE ANSWER



What are some notations that have been used to specify the syntax of programming languages?

HIDE ANSWER

BNF, EBNF, ABNF.

What syntax notation is used for official Internet standards? HIDE ANSWER

ABNF

Summary

We've covered:

- ✅ Motivation for the study of syntax
- ✅ Difference between syntax and semantics
- ✅ How to design a syntax
- ✅ Syntax diagrams
- ✅ Grammars
- ✅ Lexical vs. Phrase rules
- ✅ Ambiguity

- ✅ Precedence and associativity
- ✅ The problem of context
- ✅ Abstract syntax
- ✅ Tree grammars
- ✅ Syntax IRL
- ✅ Alternative syntax descriptions

Regular Expressions



Regular Expressions are so cool. Knowledge of regexes will allow you to save the day.

CONTENTS

Definitions • Basic Examples • Notation • Using Regular Expressions • Components of Regexes • Performance Pitfalls • Performance Tips • Miscellaneous Language-Specific Notes • Study and Practice

Definitions

In formal language theory, a **regular expression** (a.k.a. regex, regexp, or r.e.), is a string that represents a regular (type-3) language.

Huh??

Okay, in many programming languages, a **regular expression** is a **pattern** that **matches** strings or pieces of strings. *The set of strings they are capable of matching goes way beyond what regular expressions from language theory can describe.*

Basic Examples

Rather than start with technical details, we'll start with a bunch of examples.

Regex	Matches any string that
`hello`	contains {hello}
`gray|grey`	contains {gray, grey}
`gr(a|e)y`	contains {gray, grey}
`gr[ae]y`	contains {gray, grey}
`b[aeiou]bble`	contains {babble, bebble, bibble, bobble, bubble}
`[b-chm-pP]at|ot`	contains {bat, cat, hat, mat, nat, oat, pat, Pat, ot}
`colou?r`	contains {color, colour}
`rege(x(es)?|xps?)`	contains {regex, regexes, regexp, regexps}
`go*gle`	contains {ggle, gogle, google, gooogle, goooogle, ...}
`go+gle`	contains {gogle, google, gooogle, goooogle, ...}
`g(oog)+le`	contains {google, googoogle, googoogoogle, googoogoogoogle, ...}
`z{3}`	contains {zzz}
`z{3,6}`	contains {zzz, zzzz, zzzzz, zzzzzz}
`z{3,}`	contains {zzz, zzzz, zzzzz, ...}
`[Bb]rainf\*\*k`	contains {Brainf**k, brainf**k}
`\d`	contains {0,1,2,3,4,5,6,7,8,9}
`\d{5}(-\d{4})?`	contains a United States zip code
`1\d{10}`	contains an 11-digit string starting with a 1
`[2-9]|[12]\d|3[0-6]`	contains an integer in the range 2..36 inclusive
`Hello\nworld`	contains Hello followed by a newline followed by world
`mi.....ft`	contains a nine-character (sub)string beginning with mi and ending with ft (Note: depending on context, the dot stands either for “any character at all” or “any character except a newline”.) Each dot is allowed to match a different character, so both microsoft and minecraft will match.
`\d+(\.\d\d)?`	contains a positive integer or a floating point number with exactly two characters after the decimal point.

`[^i*&2@]`	contains any character other than an i, asterisk, ampersand, 2, or at-sign.
`//[^\r\n]*[\r\n]`	contains a Java or C# slash-slash comment
`^dog`	begins with "dog"
`dog$`	ends with "dog"
`^dog$`	is exactly "dog"

Notation

There are many different syntaxes for regular expressions, but in general you will see that:

- Most characters stand for themselves
- Certain characters, called metacharacters, have special meaning and **must be escaped** (usually with `\`) if you want to use them as characters. In most syntaxes the metacharacters are:

  `(` `)` `[` `]` `{` `}` `^` `$` `.` `\` `?` `*` `+` `|`

- Within square brackets, you only have to escape (1) an initial `^`, (2) a non-initial or non-final `-`, (3) a non-initial `]`, and (4) a `\`.

Using Regular Expressions

Many languages allow programmers to define regexes and then use them to:

- **Validate** that a piece of text (or a portion of that text) matches some pattern
- **Find** fragments of some text that match some pattern
- **Extract** fragments of some text
- **Replace** fragments of text with other text

Generally a regex is first **compiled** into some internal form that can be used for super fast validation, extraction, and replacing. Sometimes there is an explicit `compile` function or method, and sometimes special syntax is used to compile, such as the very common form `/.../`.

Validation

Example: find `"color"` or `"colour"` in a given string.

```
// Java
Pattern p = Pattern.compile("colou?r");
Matcher m = p.matcher("The color green");
m.find();                           // returns true
m.start();                          // returns 4
m.end();                            // returns 9
m = p.matcher("abc");
m.find();                           // returns false


# Perl
$p = /colou?r/;
"The color green" =~ $p;            # returns 1 (cuz no Perl true)
"abc" =~ $p;                        # returns 0 (cuz no Perl false)


# Ruby
p = /colou?r/
"The color green" =~ p              # returns 4
"abc" =~ p                          # returns nil


# Python
p = re.compile("colou?r")
m = p.search("The color green")
m.start()                           # returns 4
m = p.search("abc")                 # returns None


// JavaScript
const p = /colou?r/;
"The color green".search(p);        // returns 4
"abc".search(p);                    // returns -1
```

If you want to know if an entire string matches a pattern, define the pattern with `^` and `$`, or with `\A` and `\Z`. In Java, you can call `matches()` instead of `find()`.

Extraction

After doing a match against a pattern, most regex engines will return you a bundle of information, including such things as:

- the part of the text that matched the pattern
- the index within the string where the match begins
- each part of the text matching the parenthesized portions within the pattern
- (sometimes) the text before the matched text
- (sometimes) the text after the matched text

Example in Ruby:

```
>> pattern = /weighs (\d+(\.\d+)?) (\w+)/
=> /weighs (\d+(\.\d+)?) (\w+)/
>> pattern =~ 'The thing weighs 2.5 kilograms or so.'
=> 10
>> $&
=> "weighs 2.5 kilograms"
>> $1
=> "2.5"
>> $2
=> ".5"
>> $3
=> "kilograms"
>> $`
=> "The thing "
>> $'
=> " or so."
```

The same thing in JavaScript:

```
> const pattern = /weighs (\d+(\.\d+)?) (\w+)/
> pattern.exec('The thing weighs 2.5 kilograms or so.')
[ 'weighs 2.5 kilograms',
  '2.5',
  '.5',
  'kilograms',
  index: 10,
  input: 'The thing weighs 2.5 kilograms or so.' ]
```

Note how in JavaScript, the match result object looks like an array and an object.

The so-called *group numbers* are found by counting the left-parentheses in the pattern:

TODO PICTURE GOES HERE

Sometimes you need parentheses only for precedence purposes and you don't want to incur the cost of extracting a group. We have **non-capturing groups** for this purpose.

Ruby:

```
>> phone = /(((\d{3})(?:\.|-))?(\d{3})(?:\.|-)(\d{4})/
=> /(((\d{3})(?:\.|-))?(\d{3})(?:\.|-)(\d{4})/
>> phone =~ 'Call 555-1212 for info'
=> 5
>> [$`, $&, $', $1, $2, $3, $4, $5]
=> ["Call ", "555-1212", " for info", nil, nil, "555", "1212", nil]
>> phone =~ '800.221.9989'
=> 0
>> [$`, $&, $', $1, $2, $3, $4, $5]
=> ["", "800.221.9989", "", "800.", "800", "221", "9989", nil]
>> phone =~ '1800.221.9989'
=> 1
>> [$`, $&, $', $1, $2, $3, $4, $5]
=> ["1", "800.221.9989", "", "800.", "800", "221", "9989", nil]
```

JavaScript:

```
> const r = /(((\d{3})(?:\.|-))?(\d{3})(?:\.|-)(\d{4})/g;
> const m = r.exec("Call 1.800.555-1212 for info");
> m.index
7
> JSON.stringify(m);
["800.555-1212","800.","800","555","1212"]
```

Java

```
Pattern phone = Pattern.compile("(((\\d{3})(?:\\.|-))?(\\d{3})(?:\\.|-)(\\d{4})");
String[] tests = {"Call 555-1212 for info", "800.221.9989", "1800.221.9989"};
for (String s : tests) {
    Matcher m = phone.matcher(s);
    m.find();
    System.out.println("groupCount = " + m.groupCount());
    System.out.println("group(0) = " + m.group(0));
    System.out.println("group(1) = " + m.group(1));
    System.out.println("group(2) = " + m.group(2));
    System.out.println("group(3) = " + m.group(3));
    System.out.println("group(4) = " + m.group(4));
}
```

```
groupCount = 4
group(0) = 555-1212
group(1) = null
group(2) = null
group(3) = 555
group(4) = 1212
groupCount = 4
group(0) = 800.221.9989
group(1) = 800.
```

```
group(2) = 800
group(3) = 221
group(4) = 9989
groupCount = 4
group(0) = 800.221.9989
group(1) = 800.
group(2) = 800
group(3) = 221
group(4) = 9989
```

Substitution

Many languages have `replace` or `replaceAll` methods that replace the parts of a string that match a regex. Sometimes you will see a `g` flag on a regex instead of a `replaceAll` function.

```
alert("Rascally Rabbit".replace(/[RrLl]/g, "w"));
```

Components of Regexes

Character Classes

- Square brackets `[ ]` — means exactly one character
- A leading `^` negates, a non-leading, non-terminal `-` defines a range:

```
[abc]          a or b or c
[^abc]         any character _except_ a, b, or c (negation)
[a-zA-Z]       a through z or A through Z, inclusive (range)
```

- If you have a `]` in your set, put it first. Use `\` to escape.
- Java allows crazy extensions:

```
[a-d[m-p]]     [a-dm-p] (union, Java only, I think)
[a-e&&[def]]   [de] (intersection, Java only, I think)
[a-r&&[^bq-z]] [ac-p] (subtraction, Java only, I think)
```

- Other ways to say exactly one character from a set are:

```
\d              [0-9]
\D              [^\d]
\s              [ \t\n\x0B\f\r]
\S              [^\s]
\w              [a-zA-Z0-9_]
\W              [^\w]
.               any character at all, except maybe not a line
```

Groups

Defined above, in the section on extraction.

Quantifiers

Generally, 18 types:

	Eager	**Reluctant**	**Possessive**
Zero or one	`?`	`??`	`?+`
Zero or more	`*`	`*?`	`*+`
One or more	`+`	`+?`	`++`
m times	`{m}`	`{m}?`	`{m}+`
At least m times	`{m,}`	`{m,}?`	`{m,}+`
At least m, at most n times	`{m,n}`	`{m,n}?`	`{m,n}+`

Eager (Greedy and Generous) — match as much as possible, but give back

```
\w+\d\d\w+  // matches abcdef42skjhfskjfhsjdfs
            // but inefficiently
```

Possessive — match as much as possible, but do NOT give back

```
\w++\d\d\w+  // does not match abcdef42skjhfskjfhsjdfs
             // but is efficient
```

Reluctant — match as little as possible

```
\w+?\d\d\w+  // matches abcdef42skjhfskjfhsjdfs
             // efficiently, yay!
```

Backreferences

Things captured can be used later:

```
<(\w+)>[^<]*</\1>
```

Anchors, Boundaries, Delimiters

Some regex tokens do not consume characters! They just assert the matching engine is at a particular place, so to speak.

- `^` : Beginning of string (or line, depending on the mode)
- `$` : End of string (or line, depending on the mode)
- `\A` : Beginning of string
- `\z` : End of string
- `\Z` : Varies a lot depending on the engine, so be careful with it
- `\b` : Word boundary
- `\B` : Not a word boundary

Read more about these at Rexegg.

Also, the lookarounds (up next!) don't consume any characters either!

Lookarounds

- Lookarounds do not consume anything

- Even though they have parens, they do not capture
- **Positive Lookahead**: Matches only if followed by something

```
Hillary(?=\s+Clinton)
```

matches the Hillary in Hillary Clinton but not the Hillary in Hillary Makasa.

- **Negative Lookahead**: Matches only if not followed by something

```
q(?!u)
\b(?:[a-eg-z]|f(?!oo))\w*\b      // Word not starting with foo
\b(?:[a-eg-z]|f(?!oo\b))\w*\b    // Any word except foo
((?!foo).*)
```

- **Positive Lookbehind**: Matches only if preceded by something

```
(?<=-)\p{L}+              // a word following a hyphen
(?<=http://)\S+           // URL not including the http:// part
```

- **Negative Lookbehind**: Matches only if not preceded by something

```
(?<![-+\d])(\d+)      // Digits not preceded by a digit, +, or -
```

- Lookarounds show up in search and replace applications

```
Pattern p = Pattern.compile("Hillary(?=\\s+Clinton)");
String text = "Once Hillary Clinton was talking about Sir\n" +
    "Edmund Hillary to Hillary Makasa and then Hillary\n" +
    "Clinton had to run off on important business.";
Matcher m = p.matcher(text);
System.out.println(m.replaceAll("Secretary"));
```

Note: Read this awesome article on lookarounds.

Modifiers

A modifier affects the way the rest of the regex is interpreted. Not every language supports all of the modifiers below. For example, JavaScript (officially) supports only i, g, and m.

Modifier	Meaning
`g`	global
`i`	ignore case
`m`	multiple line
`s`	single line (DOTALL): Means that the dot matches any character at all. Without this modifier, the dot matches any character except a newline.
`x`	ignore whitespace in the pattern
`d`	Unix line mode: Considers only U+000A as a line separator, rather than U+000D or the U+000D/U+000A combo or even U+2028.
`u`	Unicode case: in this mode the case-insensitive modifier respects Unicode cases; outside of this mode that modifier only consolidates cases of US-ASCII characters.

Performance Pitfalls

You should know some things about how your *regex engine* works since two "equivalent" regexes can have drastic differences in processing speed.

- It is possible to write regexes that take exponential time to match, but you pretty much have to TRY to make one (they're pathological)
- It is more common to accidentally create regexes that run in quadratic time
- Main types of problems
    - Recompilation (from forgetting to compile regexes used multiple times)

      ```
      // Java shortcut, should not be used in most cirumstances
      s.matches("colou?r");
      ```

    - Dot-star in the Middle (which causes backtracking)
        - Solution 1: Use negated character class
        - Solution 2: Use reluctant quantifiers
    - Nested Repetition

Performance Tips

Always do the following:

- Use non-capturing groups when you need parentheses but not capture.
- If the regex is very complex, do a quick spot-check before attempting a match, e.g.
  - Does an email address contain '`@`'?
- Present the most likely alternative(s) first, e.g.
  - `black|white|blue|red|green|metallic seaweed`
- Reduce the amount of looping the engine has to do
  - `\d\d\d\d\d` is faster than `\d{5}`
  - `aaaa+` is faster than `a{4,}`
- Avoid obvious backtracking, e.g.
  - `Mr|Ms|Mrs` should be `M(?:rs?|s)`
  - `Good morning|Good evening` should be `Good (?:morning|evening)`

Miscellaneous Language-Specific Notes

A few things that are good to know:

- Java's built-in support for regexes exceeds that of many languages
- Especially good for Unicode and for character classes (has more than Perl)
- Syntax is more cumbersome (string literal support weak, no operator for matching...) — live with it!
- Perl has nice regex, too, even allows you can even embed code inside them.
- Great Perl documentation at the perldoc pages perlrequick, perlretut, perlre. and perlreref.
- JavaScript seems to less extensive support than other languages, but I think this is changing.
- Python puts regex functions in a module.
- Python docs are here.

Study and Practice

Here are some good sources:

- The Premier Site for Regexes
- Maybe this is a better premier site
- Regex 101—Develop regexes online
- RegExr (Awesome online tool for Java regexes)
- Rubular (Ruby Online Regex Tester)
- Perl Regular Expressions Tutorial
- Perl Regular Expressions (Manual)
- Perl Regular Expressions Quick Reference
- Ruby Regexp class documentation
- Java Regex Tutorial
- Java Regex Optimization Article
- Java Pattern class API docs
- JavaScript Regular Expressions (at Mozilla Developer Center)
- Python Regular Expressions

Introduction to Parsing Theory

Compilers need to parse. A parser is just code, so to build one, we could just wing it. But a theory helps us understand parsing at a deep level, enabling us to write better, faster, and provable correct parsers. Let's learn!

CONTENTS

What is Parsing Again? • Lexical Analysis • Approaches to Parsing • Recursive Descent • Table-Based Parsing • Parsing Expression Grammars • Real-World Issues • Resources • Summary

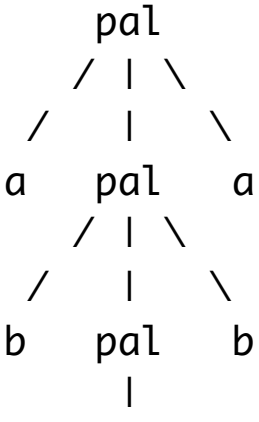
What is Parsing Again?

Parsing is the process of uncovering the derivation, or derivation tree, of a string from a grammar.

For example, given the grammar:

pal ⟶ ("a" | "b" | "a" *pal* "a" | "b" *pal* "b")?

we **parse** the string ***abbba*** into the derivation tree:



|
b

In a real programming language, we generally don't make a derivation tree for each character in the source program. Instead, we first group characters into **tokens** and we consider the tokens as the symbols.

TODO

The pre-processing of the source program via tokenization is so common and so natural and so important that there is a kind of separate (simpler) theory and practice that deals with this, called **lexical analysis.** Let's briefly talk about that first before getting into the richer theory of parsing.

Lexical Analysis

Tokenization, or lexical analysis, is the process of forming tokens out of the source code character stream. You basically look at the character stream character-by-character, skipping whitespace and comments, and collecting characters into tokens. Sometimes you might need to "look ahead" a little, or "back up" as you go through the character stream.

Generally, each token will have a type, a lexeme, and line and column position. Types of tokens include:

- keyword
- identifier
- numeric literal
- string literal
- punctuation

The **lexical grammar** is the part of the language specification that shows how tokens are formed from characters. Negative lookahead is common! An example might be:

TODO

Sometimes people find it helpful to visualize how a **lexer** works via a Finite Automata:

TODO

Writing a lexer by hand is possible. For the simple lexical grammar shown above, a hand-crafted lexer might be:

TODO

In general, lexers are concerned with issues such as:

- Case sensitivity (or insensitivity)
- Whether or not blanks are significant
- Whether or not newlines are significant
- Whether comments can be nested

Errors that might occur during scanning, called **lexical errors** include:

- Encountering characters that are not in the language's alphabet
- Too many characters in a word or line (yes, such languages do exist!)
- An unclosed character or string literal
- An end of file within a comment

Once you have a lexer written and deployed, you can now work entirely with token streams. Parsing now works entirely at the level of the phrase grammar. The parser's job is to uncover the derivation tree, or parse tree, whose leaves are now tokens, not characters.

Approaches to Parsing

A language's syntax is normally defined with a CFG (or the equally powerful BNF, EBNF, Syntax Diagram), or a PEG. A compiler uses the grammar to **parse** the token sequence, that is, *to determine if the token string can be generated by the grammar, and produce the syntax tree for the token sequence*.
People have been studying parsing for decades! Here are some things they figured out:

- Context-sensitivity is too hard. Separate it out. Just focus on the context free parts of the language in the parser.
- There's a desire for grammars for real programming languages to be **unambiguous**.
- Parsers should be fast! There is a theorem that says every CFG can be parsed in $\Theta(n^3)$ time. That is too slow. Perhaps we should look for restricted CFGs that can be parsed more efficiently.

- There is! People have figured out and defined the classes of LL grammars and LR grammars! These are restricted CFGs that can be parsed in $\Theta(n)$ time. Slay!
- Massive amounts of theory have been generated around parsing techniques for LL and LR grammars, since the mid-20th century. LL mechanisms are easy but the class of LL languages is much smaller than LR. But LR methods are really tough to follow and not meant for human understanding.
- PEGs hit the scene in 2004 and are awesome! They are a little different from CFGs. They can be parsed in linear time with a packrat parser, but at the cost of a lot of additional space. Ohm adds some power to PEGs (left recursion) so you *might* encounter super-linear parse times.

That's some good background info to start a study of parsing. But wait, first, what do LL and LR mean?

$LL(k)$ Grammar

A grammar that can be used to drive a leftmost derivation by reading the input left-to-right, examining only the next k symbols, and never backing up.

$LR(k)$ Grammar

A grammar that can be used to drive a rightmost derivation by reading the input left-to-right, examining only the next k symbols, and never backing up.

Um, are there more practical descriptions of these language classes? Pretty much, yeah. Try these:

$LL(k)$

Locate all the "choice points" in the grammar (these are easy to find in the equivalent syntax diagram). Determine the maximum number of tokens, k, required to know exactly which path to take over all choice points. If k is finite, the grammar is LL(k).

$LR(k)$

The set of LR languages is exactly equivalent to the set of languages recognizable by a DPDA. LR languages are exactly the DCFLs. An $LR(k)$ grammar is a CFG for a language that is a DCFL.

An LL and LR Fact Sheet:

LL Grammars	LR Grammars
• Subset of CFGs • Never ambiguous • Parsable in Linear Time	• Subset of CFGs • Never ambiguous • Parsable in Linear Time

• Left-to-right, Leftmost derivation • Top-down • Predictive • Expand-Match • Parsers can be hand-made • Can give nice error messages • Can not handle left recursion • Less "powerful" than LR	• Left-to-right, Rightmost derivation • Bottom-up • Not Predictive • Shift-Reduce • Nearly impossible to write parsers by hand • Must work hard to give good errors • Left recursion is desirable! • More "powerful" than LL

Wow! That was a lot. Details are coming. Read on.

> ***Exercise**: Why is it that a grammar with left-recursion can not be LL(k) for any k?*

> ***Exercise**: For more on LL and LR see this BNF article with a great little section on parsing.*

We've made syntax diagrams to describe all the legal programs in the language, but how to we follow the diagrams to "match" or "parse" a candidate program? That is, given a program (a candidate string), how, exactly, do we figure out how it is matched by, or generated by, the syntax diagrams? How do we **parse**?

There have been hundreds of books and thousands of papers on the topic of parsing over the years. Here we just need to highlight a few things.

Hand-Crafted Parsers

Because a parser is just a program that turns source code into a parse tree, you can just write on by hand. The most common kind of hand-crafted parser uses the technique of recursive descent, which you can read about at Wikipedia. You kind of walk through each of the syntax diagrams.

CLASSWORK

I've written a recursive descent parser as part of an Astro compiler. We'll take a very brief look at it in class.

Parser Generators

A recursive descent parser generally requires a highly restricted syntax, one that is line with a class of languages known as the $LL(k)$ languages. These languages are a proper subset of the $LR(k)$ languages (also known as the deterministic context-free languages), which are used in popular parser generators such as YACC, Bison, and ANTLR, among others. While these systems are quite popular and famous, we won't be covering them here. Do read about them, though.

We will see $LL(k)$ and $LR(k)$ later in the course, though.

Analytic Grammars

Another approach is to use a specialized kind of grammar designed specifically for parsing. So instead of using traditional **generative grammars**, covered previously in our notes on Language Theory (which specify a language by giving a set of rules to **generate** all and only those strings in a language), we can use an **analytic grammar**, in which the rules specify how to **match** a candidate string.

The biggest difference between generative and analytic grammars is in the handling of alternatives. In a generative grammar G, $L(G)$ is contains all strings that *can* be generated by applying rules whenever possible. For example, the rule $S \rightarrow A \mid B$ means you can replace an S with an A or a B, so you can generate both. So the grammar:

```
s ⟶ "a" | "ab"
```

describes the language $\{a, ab\}$.

In an analytic grammar, the rules are deterministic; you don't get to pick one or the other. So $A \ / \ B$ means: "Try to match A. If you can, the rule matches and you are done. Otherwise (and only otherwise), try to match B". So:

```
s ⟵ "a" / "ab"
```

recognizes the language $\{a\}$, but:

```
s ⟵ "ab" / "a"
```

recognizes the language $\{a, ab\}$.

Note the tradeoff here. Determinism means there's never ambiguity (a string can only parse one way, if it can at all), which is great! But the ordering of the rules matters, so you have to be careful to

do what you want. Here's another illustration. If we wanted an analytic grammar for Astro, the rule:

$$\mathit{Primary} \longleftarrow \mathit{num} \;/\; \mathit{id} \;/\; \mathit{Call} \;/\; \ldots$$

would be wrong, as it will *never* match a call!

***Exercise**: Why, exactly, must the Call alternative precede the id alternative?*

CLASSWORK

This might be deep, so we'll try a few more examples.

Unlike generative grammars, analytic grammars feature special **lookahead operators**, including the **negative lookahead operator** (~) and the **positive lookahead operator** (&). This enables the definition of non-context-free languages, despite having only one symbol on the left-hand side! Here's an analytic grammar for the non-context-free language $\{a^n b^n c^n \mid n \geq 0\}$:

$$\begin{array}{lcl} s & \longleftarrow & \texttt{\&}\,(x\ \texttt{"c"})\ \texttt{"a"}^{+}\ y \\ x & \longleftarrow & \texttt{"a"}\ x?\ \texttt{"b"} \\ y & \longleftarrow & \texttt{"b"}\ y?\ \texttt{"c"} \end{array}$$

Parsing Expression Grammars (PEGs)

There are various formalisms of analytic grammars (Wikipedia identifies several); the most popular is the Parsing Expression Grammar (PEG). To read a PEG description, pronounce the operators as follows:

- `A B` : match A then match B
- `A / B` : match A. If it matched, success. Otherwise, match B
- `A?` : match A zero or one times
- `A*` : match A zero or more times
- `A+` : match A one or more times
- `&A` : match A without consuming it (but it must match)
- `~A` : match A without consuming it. If A matches then ~A fails. If A fails ot match, then ~A succeeds
- `end` : matches the end of the string
- Quoted text: successively match each symbol exactly

Note how PEGs use `/` instead of `|`, perhaps to emphasize the deterministic nature of matching.

Strictly speaking, it should be impossible in a PEG to have **left-recursive rules** like:

Exp ⟵ ***Exp*** **"+"** ***Term*** / ***Term***

given that "in order to match an Exp, you much first try to match an Exp." It's sounds circular. In fact it is, and traditionally, PEG implementations suggested using the rule:

Exp ⟵ ***Term*** **("+"** ***Term*****)***

instead. However, it turns out a proper parsing technique (using a **packrat parser**) can deal with it! It's not easy, but you can read the original paper by Alex Warth et al or this nice and detailed article by Guido van Rossum to see how it's done.

> ***Check your understanding***
>
> *Analytic grammars:*
>
> - ✅ *Are designed for parsing (recognition) rather than generation of languages.*
> - ✅ *Are written with only a single variable on the left hand side of each rule.*
> - ✅ *Use ⟵ rather than ⟶ to define rules.*
> - ✅ *Execute alternatives left-to-right and once you get a match, you keep it, even if it leads to a dead end.*
> - ✅ *Have powerful lookahead operators, thanks to deterministic execution.*
> - ✅ *Can handle left-recursive rules.*

Try this exercise before moving on:

***Exercise**: Show that the following generative grammar for assignment statements and if-then-else statements in a programming language:*

```
S → "a" | "i" "x" "t" S ("e" S)?
```

is ambiguous, but that the ambiguity can be resolved by rewriting as a PEG:

```
S ← "a" / "i" "x" "t" S "e" S / "i" "x" "t" S
```

How, exactly, is the ambiguity resolved? (Historical note: this is the dangling else problem, a classic in parsing theory.)

Now let's put everything together to come up with a Parsing Expression Grammar for Astro, that (1) properly separates lexical and phrase rules, (2) deals properly with precedence and associativity, (3) embraces left-recursion, and (4) correctly handles reserved words.

Program	⟵	*Statement*$^{+}$
Statement	⟵	*Assignment* / *PrintStmt*
Assignment	⟵	*id* `"="` *Exp* `";"`
PrintStmt	⟵	*print Exp* `";"`
Exp	⟵	*Exp* (`"+"` | `"−"`) *Term* / *Term*
Term	⟵	*Term* (`"*"` | `"/"` | `"%"`) *Factor* / *Factor*
Factor	⟵	*Primary* (`"**"` *Factor*)? / `"−"` *Primary*
Primary	⟵	*num* / *Call* / *id* / `"("` *Exp* `")"`
Call	⟵	*id* `"("` (*Exp* (`","` *Exp*)*)? `")"`
num	⟵	*digit*$^{+}$ (`"."` *digit*$^{+}$)? ((`"E"`|`"e"`) (`"+"`|`"−"`)? *digit*$^{+}$)?
print	⟵	`"print"` ¬*idchar*
id	⟵	¬*print letter idchar**
idchar	⟵	*letter*| *digit*| `"_"`
space	⟵	`" "`|`"\t"`|`"\r"`|`"\n"`|`"//"` (¬`"\n"` *any*)*

> ***Lexical and Phrase Variables***
>
> *Ohm also adopts the convention of starting the names of lexical variables with a lowercase letter and phrase variables with an uppercase letter.*

Recursive Descent

Table-Based Parsing

Parsing Expression Grammars

PEGs are designed for direct parsing: there's no need to "extract" any kind of machine. The matching process is built into the grammar rules.

TODO

Must reads:

- Wikipedia's article
- The original paper by Bryan Ford
- Short blog post on CFGs vs PEGs
- Using PEGs Derivatives (Paper that pointed out a huge bug in Ford's original paper)
- 2014 paper relating CFGs and PEGs
- 2020 paper on The computational power of PEGs

Real-World Issues

The language designer will have to take into account any ambiguities in the language description. For example, in C, the expression `(x)-y` hase two different syntactic interpretations: if `x` refers to a type, then its a cast of a negation, otherwise it is a subtraction. You can't tell at this phase.

Errors that occur during parsing, called **syntax errors** arise from malformed token sequences such as:

- `j = 4 * (6 - x;`
- `i = /5`
- `42 = x * 3`

TODO: non-syntax errors

Resources

Summary

We've covered:

- ✅ What parsing is
- ✅ Lexical analysis vs. syntax analysis
- ✅ Various approaches to parsing
- ✅ The method of recursive descent
- ✅ Parsing based on Context-Free Grammars
- ✅ A good deal about Parsing Expression Grammars
- ✅ Where to learn more